A plan for proving Ochr consistent

The rough argument to be followed, the lemmas it needs, and where it could fail

Draft for discussion · 2 November 2026 · a plan for a proof, not a proof

Summary. The plan proves consistency of Ochr the way the paper proves soundness of its first-order fragment, extended to the whole calculus. Types are given meanings in a set model, in which `False` denotes the empty set. A soundness theorem says that whatever an accepted program proves is true in the model at every concrete instance; it is proved by a simulation between the checker's symbolic run and the concrete machine, by induction on the program. Consistency is its corollary. The simulation rests on one family of lemmas, the agreement of the two evaluation paths (stability), which this document splits into named lemmas with dependencies and a status for each. Of the seven decisions in its decision half, two are proved in Lean against the checker, a third in part, and a fourth is reduced to a smaller lemma. The core lemmas of its run half, parametricity (a run cannot depend on an unknown it never inspects) and the frame property (a call affects only what it is passed), are proved on a re-engineered model of the machine (the heap slice), and the correctness of closing off (how the checker turns a call stuck on an unknown into a value) in part; none of the run half is proved for the checker as it stands. Eight design questions must be settled before the full simulation can be written down; they are marked where they arise and collected in Section 10.

Contents

1 What this document is 3
2 The calculus, as far as the plan needs it 3
2.1 Terms, values and neutrals 3
2.2 States, borrows and resolution 4
2.3 The machine, closing off and sealed programs 4
2.4 Observation, footprints, `Id` and `Eq` 5
2.5 Erasure and private copies 5
2.6 Typing 5
2.7 Conversion 6
2.8 Refinements, valuations and the two paths 7
3 The target 7
4 The shape of the argument 8
4.1 The set model 8
4.2 Conversion is sound 14
4.3 The soundness theorem 14
4.4 How soundness is proved: the walk 15
5 The principles the checker enforces 17
5.1 Locality 17
5.2 Closing off 18
5.3 Worlds 18
5.4 Wholeness 19
5.5 Erasure and captures 19
5.6 Recursion 20
5.7 Observation 20
6 The simulation relation, beyond the fragment 21
6.1 The fragment F 21
6.2 Agreement and its assumptions, in F 21

6.3 Agreement for the whole calculus, proposed . . . 22
6.4 Naturality, generalised . . . 22
6.5 Feature by feature . . . 22
7 The lemmas and how they depend on each other . . . 25
7.1 The walk's lemmas . . . 25
7.2 Stability: the two paths agree . . . 26
7.3 The dependency graph . . . 29
8 The heap port, and the order of work . . . 30
8.1 Why a port is on the critical path . . . 30
8.2 What does not depend on the port . . . 31
8.3 What does . . . 31
8.4 Recommended order . . . 31
9 Where the plan could fail . . . 32
10 Open design questions, collected . . . 33
11 Referee reports . . . 34
11.1 Foundations and semantics . . . 34
11.2 The simulation and stability . . . 35
11.3 The structure of the argument and its targets . . . 36

1 What this document is

This is the plan we intend to follow to prove that Ochr is consistent: the shape of the argument, the definitions it needs, the lemmas it reduces to, which of those are already proved and where, and where the plan could fail. It is written as a section of theory work would be, so the statements are precise where they can be, and where they cannot be yet, that is said.

The object of study. The calculus is the one implemented by the Lean checker in `ochr/core/lean/Ochr/` (chiefly `Syntax.lean`, `Basic.lean`, `Env.lean`, `Machine.lean`, `Decide.lean`, `Obs.lean`, `Check.lean` and `Run.lean`) and laid out rule by rule in the appendix of the Ochr paper. Section 2 restates every part of it that the plan relies on, so that the plan can be followed without either.

Status labels. Each lemma carries one of five labels.

- **proved in Lean**: proved, without `sorry`, in Lean against the checker's own functions. The Lean name is given.
- **proved on the heap slice**: proved, without `sorry`, in Lean for the *heap slice*, a second evaluator for part of the calculus built in the style the checker would have after the port described in Section 8. Not yet a statement about the checker.
- **reduced to X**: shown to follow from the named lemmas X by an argument given here.
- **paper argument**: an argument exists at the level of detail of this document, not checked in Lean.
- **open**: no argument yet.

Design questions that must be decided before an argument can be written are marked **open design question**.

Route. Of the routes to a correctness story we have compared, this plan follows the *operational simulation* route (show that every step of the checker's symbolic run is matched by the concrete machine at every instance) combined with a *denotational model* (assign every type a mathematical meaning): the model says what "true" means, and the simulation shows that the checker only accepts what is true. A different route, translating each accepted program into Lean and having Lean's kernel check it, gives consistency one program at a time; it is not part of this plan, though its results are used in Section 9 as evidence.

2 The calculus, as far as the plan needs it

2.1 Terms, values and neutrals

Terms and types share one grammar: variables, the universes `Prop` and Type_i, n-ary Π-types, recursive functions **fix** $f\left(\overline{x} : \overline{A}\right) : B$ **by** $x_j \coloneqq t$ (a `fix` without `by` is a λ), saturated calls, inductive types $\mathsf{D}(\overline{a})$ and constructors $\mathsf{C}(\overline{t})$, the primitive equality `Eq` $A\ t\ u$, transport $\mathsf{J}(A, a, b, P, h, t)$ and **rewrite** h **in** t, borrow types $\&A$, places $p \coloneqq x \mid *p \mid p.g$, borrowing $\&p$, assignment $p \coloneqq t$, `let`, sequencing $t; u$, **match** p $\{\mathsf{C}_1(\overline{y}_1) \Rightarrow t_1, ...\}$ with any number of arms, **split** f **in** t, and `Id` $A\ t\ u$, the equality of two computations.

Every type of data and every connective is an inductive declaration **inductive** $\mathsf{D}\ \left(\overline{x} : \overline{X}\right) : s \coloneqq \mathsf{C}_1\left(\overline{y}_1 : \overline{B}_1\right) \mid ...$ with $s \in \{\mathsf{Prop}, \mathsf{Type}_0\}$. The library declares `Nat`, `Unit`, `Pair`, `False` (no constructors), `True` (one constructor `I`, written `refl`) and `And`. A field's type may mention the type being declared, other declared types, parameters, earlier fields of the same constructor (the fields form a *telescope*), calls of earlier *type functions* (definitions whose codomain is a sort), and equations between *first-order data* (types built from inductive types and parameters, with no Π, no `&` and no sort); never `Π` or `&`. Parameters whose declared type is syntactically a sort are *type parameters*; the others are *value parameters*, and a recursive occurrence of `D` in a field may take any arguments at them (*non-uniform* occurrences). A family indexed by values is written by fixing the index with an equation field (*Fording*): `Vec(A, n) := VNil(h : Eq(Nat, n, Z)) | VCons(m : Nat, x : A, xs : Vec(A, m), h : Eq(Nat, n, S(m)))`.

$\&A$ requires $A : \mathsf{Type}_0$, and `&` occurs only as the whole declared type of a parameter, result or annotated term: never inside another type and never as the result of computing a type. So there are no borrows inside data and no borrows of borrows. Universes are not cumulative, and `Prop : Type₁` beside `Type₀ : Type₁`.

Values are constructor values $\mathsf{C}(\overline{a};\overline{v})$ (recording the inductive's parameters $\overline{a}$), the proof value $\star$, top-level function names, closures $\langle\overline{\kappa}\vdash \mathbf{fix}\ ...\rangle$ (code with captured values $\overline{\kappa}$), Π-closures $\langle\overline{\kappa}\vdash \Pi\big(\overline{x}:\overline{A}\big).B\rangle$ (types are values), type values, borrows $\mathrm{borrow}_\ell v$, loans loan_ℓ, the moved-out marker $\bot$, and *neutrals*. A neutral is an *abstract value* σ (an unknown of a known type $\Delta(\sigma)$, introduced only by the checker) or a *sealed program* $\lceil t\rceil$ (a program with no free variables whose run is stuck). A value is *borrow-free* if it contains no borrow and no live loan. A value is *closed* if it contains no free variables, no abstract values, no loans and no $\bot$: nothing whose meaning depends on an environment, on what the unknowns stand for, or on contents held elsewhere in the state. A closed value therefore means the same thing wherever it appears, which is what lets the model give it a meaning $[\![v]\!]$ on its own (Section 4.1). A sealed program has no free variables but is stuck on some abstract value, so it is not closed; nor is anything containing one.

2.2 States, borrows and resolution

An environment Ω is a stack of frames, each binding variables (with declared types) to values, plus temporaries. A *state* adds Δ (the types of abstract values), ρ (the refinements and generalisation records made so far), the goal while a definition is checked, and the recursion stack. $\mathrm{content}_\Omega(p)$ follows a place through variables, borrows ($*\,p$ looks inside $\mathrm{borrow}_\ell v$ at v) and fields.

Borrows. $\&p$ moves the content v of p into $\mathrm{borrow}_\ell v$ and leaves loan_ℓ at p. *Ending* the borrow ([End]) replaces it by $\bot$ and substitutes its content for every occurrence of loan_ℓ: a loan is a variable bound by its borrow. Every access to a place is preceded by [Access], which ends every borrow whose loan lies on the path to the place or inside its content (for a match, which inspects only the head: on the path, at the head, or anywhere inside a neutral head). Reading non-copy data moves it out, leaving $\bot$; data of a *copy type* (`Unit`, sorts, propositions, inductives declared `copy`, and non-recursive inductives whose fields are copy types) is copied, and $\mathbf{clone}(p)$ copies explicitly. The machine's rules for these steps are [Borrow], [Read] (moving data out), [Move] (moving a borrow), [Copy], [Clone] and [Assign]; [Ctor] builds a constructor value. A borrow must be whole (no $\bot$ inside) when it ends, is returned, or is passed to a call that closes off.

Dropping. When an owned value dies holding a live loan, [Drop] ends the borrower if the borrower is held in a binding, and fails if the borrower is a value in flight (a temporary such as a result being returned). So a local may die while a variable that borrowed it is never used again, but nothing may return a borrow of its own local.

Owners and resolution. $\mathrm{owners}_\Omega(\ell)$ follows each occurrence of loan_ℓ outward: through the content of an enclosing borrow_m to $\mathrm{owners}_\Omega(m)$, otherwise to the owned position holding it. The *resolution* $\rho(\Omega)$ ends every borrow; $\rho_\Omega(w)$ is w with each live loan replaced by the resolved content of its borrow.

Well-formedness. A state is well formed when (WF1) each label has at most one borrow, which is the whole value of a binding or temporary, never inside data, a sealed program, a closure or a type; (WF2) every loan's borrow is in Ω, except a loan inside a sealed program whose borrow is outside the run (an *inert* loan); outside sealed programs a label occurs as a loan at most once, inside them any number of times; (WF3) the relation "a loan of ℓ occurs in the content of borrow m" is acyclic; (WF4) closures and Π-closures capture no borrow and no $\bot$, and sealed programs are closed; (WF5) at every call point each argument is loan-free or is $\mathrm{borrow}_\ell u$ with u loan-free; (WF6) each binding's value is $\bot$, or a loan whose borrow's content has the binding's type, or a value of the binding's type, and every σ has a type in Δ.

2.3 The machine, closing off and sealed programs

The machine is big-step and deterministic: $\langle\Omega,t\rangle \Downarrow \langle\Omega',v\rangle$, or the run is *stuck* (it must inspect a neutral), or it fails. Calls evaluate their arguments left to right into temporaries, push a frame binding the parameters, run the body, and pop the frame, dropping its bindings. Types and proofs are *erased*: an erased term runs on a private copy of the state, which is discarded (Section 2.5).

Closing off ([Close]). When a call $f(\overline{w})$ is stuck, the call itself returns. Let the borrow arguments be $w_i = \mathrm{borrow}_{\ell_i} u_i$ ($i \in I$) with each u_i loan-free, let L be let $c_i = u_i$ for $i \in I$, and C the call $f(\overline{a})$ with $a_i = \&c_i$ for $i \in I$ and $a_i = w_i$ otherwise. If f's declared codomain is borrow-free, the call returns $\lceil L; C\rceil$ and each loan_{ℓ_i} is filled with $\lceil L; C; c_i\rceil$. If it is $\&T$, the call returns $\mathrm{borrow}_k\lceil L;\ \mathrm{let}\ r = C;\ *r\rceil$ for a fresh k, and each loan_{ℓ_i} is filled with $\lceil L; \mathrm{let}\ r = C; *r := \mathrm{loan}_k; c_i\rceil$: the fills have a *hole* loan_k for the final content the caller will write through the

returned borrow, which the caller's eventual [End] of borrow_k supplies by substitution. A result at type `Unit` is `()`. A call whose head is a neutral is stuck at once and closed off the same way.

Sealed programs. Values are kept in normal form. A sealed program's normal form is its run from the empty environment, except that its head call C may not itself close off ([Seal]'s head guard); so a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run is inert: the run treats it like an abstract value.

Stuck blocks. A stuck `match` that is not the body of a call is closed off as a call of an anonymous function of the places it uses. Pattern variables are first resolved to the places they denote; the block takes each maximal place by move if some arm moves it, by borrow if some arm writes or borrows it, and by copy otherwise. A stuck `J` is closed off the same way, with its transported term as the block's one arm.

2.4 Observation, footprints, `Id` and `Eq`

The *footprint* $W_\Omega(t, u)$ is the set of owners of the roots of the places that t or u borrows or assigns, and of the borrow variables they read whole, in a canonical order read off the syntax. A place that is only read contributes nothing. The owner of a variable holding a borrow is the owners of its loan; of an owned variable, the variable itself.

The *observation* $\text{Obs}^W_\Omega(t)$ runs t on a private copy of Ω, ends every remaining borrow, and returns the tuple of the result and the contents of the positions of W. A borrow-typed result is observed after writing a given fresh σ_w through it, so that its owners show where it points ([Obs-borrow]); only conversion observes such results.

$\mathsf{Id}(A, t, u)$ computes, at Ω, to $\text{and}(\text{eq}(A, r, r'), \text{eq}(T_1, w_1, w_1'), ...)$, where $\text{Obs}^W_\Omega(t) = (r, \overline{w})$, $\text{Obs}^W_\Omega(u) = (r', \overline{w'})$ and $W = W_\Omega(t, u)$; A must be borrow-free. `Eq` computes through the smart constructor `eq`: on two values of one constructor it is the conjunction of the fields' equations (injectivity; for a *dependent* constructor, one some of whose fields' types mention earlier *index* fields, only when the index fields are convertible), on values of different constructors of one inductive it is `False` (disjointness), on convertible values (equal by definitional equality, Section 2.7) it is `True`, and otherwise, or whenever $\bot$ occurs in a side, it is the stuck type `Eq` T v w. `and` drops `True` conjuncts. $\mathsf{J}(A, a, b, P, h, t)$ computes to t only when $a \equiv b$; otherwise it is stuck and closed off.

2.5 Erasure and private copies

A term's *declared type* is read from the declared types of its heads without normalising (a variable's declared type, a call's callee codomain term, a constructor's inductive, an annotation, a tail's or arms'). A term's *declared sort* is the sort of its declared type, read the same way. A term is *erased* when it is a proof (its declared type has declared sort `Prop`) or forms a type (a type former, a call of a function returning a sort, or anything in a *type position*: an annotation, a Π domain or codomain, a type slot of `Eq`, `Id` or `J`, or a parameter or field type). An erased term is a *statement*. A run is in *erased mode* while it runs a statement, and in *runtime mode* while it runs code. A Π-type's *class* (returns types, returns proofs, other) and whether it returns a borrow are read from its codomain term when it is formed, and Π-types that differ in either are not convertible. *Sorts are syntactic*: whatever is written in a type position must have a declared type that is syntactically a sort, so a type's declared sort is its computed sort. Every erased term runs on its own private copy of the state, which is discarded. In the checker these decisions are functions of syntax, declarations and declared types only, collected in one module (`Decide.lean`) that cannot run the machine.

2.6 Typing

The typing judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ is the machine run with types: each rule performs the machine's step and computes a type. Where the machine would be stuck on the program being checked, typing splits instead. The rules the plan relies on:

- **[Call-type].** The type of a call is its callee's codomain evaluated, on a private copy, with the parameters bound to the arguments' values; each argument's type must convert to the corresponding parameter type, evaluated the same way.
- **[T-Call].** The call's type is its [Call-type]; then the callee runs in the machine, where a stuck body closes off. For a proof-typed call ([T-Call-proof]) the body is never run.

- **[Rec].** In the body of **fix** f... **by** x_j, f occurs only as the head of a call (also inside nested functions and block arms), and every call of f, recognised by the name f binds for itself or a nested function's capture of that name, passes in position j a strict subterm (through any field, and through the borrow for &D) of the *entry value* of x_j (its value when the call started), as refined so far.
- **[Split].** A match on a place whose content is an abstract value σ checks each arm i under the refinement $\sigma := \mathsf{C}_i(\overline{\sigma}_i)$ with fresh field values typed by the constructor's telescope; the refinement is applied to the environment, every stored type, Δ and the goal, with sealed programs re-normalised. Then the match is closed off as a stuck block, and checking continues once from the unrefined state.
- **[Split-gen].** A match on a sealed program n first *generalises* it: replace n everywhere by a fresh σ of the place's type, record $n := \sigma$ (so a later derivation of n normalises to σ), then split σ. **split** f **in** t does the same for the first stuck call of f in the goal.
- **Worlds.** Each arm of a split is checked in its own *world*. Leaving an arm discards everything it established except fresh-name counters and the types of abstract values it created. A generalisation record, and an η-*refinement* (refining σ to $\mathsf{C}(\overline{\sigma})$ because σ's type has only the one constructor C), lives until the arm that made it ends: the constructor follows from the type alone, but the fields' types are the arm's.
- **[Def].** A definition is checked at its *generic call*: each borrow parameter is bound to $\mathrm{borrow}_{\ell_i}\sigma_i$ with a fresh owned cell $c_i \mapsto \mathrm{loan}_{\ell_i}$ in a bottom frame, each proof parameter to $\star$, each other parameter to a fresh σ_i. The goal G is the [Call-type] of that call. The body is checked in *tail position* (where only trailing drops follow: the body itself, and the tails of `let`s, sequences and arms in tail position), producing a finite *case tree* whose leaves (Ω', v, A) are the states, values and types at the end of each path; on every leaf the frame's pop must succeed and $A \equiv G'$, the goal refined along that path. The tail rules are [Tail-let] and [Tail-seq] (run a `let` or sequence, then continue), [Tail-match] (a match on a known constructor), [Tail-split] (a split on an abstract value, one subtree per arm), [Tail-gen] (generalise a sealed program, then split), [Tail-prop] (a match on a proof, one subtree per arm) and [Tail-end] (a leaf). Constants are checked from the empty state ([Const]); inductive declarations by [Ind-decl], which checks the conditions of Section 4.1.3.
- **Matches on proofs.** A match whose arms name constructors of an inductive in `Prop` is decided by the scrutinee's type, never its content ($\star$). It may produce data or a type only if the inductive is a *subsingleton* (no constructors, or one whose fields are all proofs). `match h {}` on h : `False` is ex falso; in tail position it contributes no leaves.

> **Quick explainer: large elimination and subsingletons.** *Eliminating* a value means taking it apart with a match. Matching on data is unrestricted. Matching on a proof to produce data or a type is called *large elimination*, and it needs care, because proofs are erased and every two proofs of one proposition count as equal (*proof irrelevance*). If the result could depend on which constructor the proof used, two "equal" proofs would give different results. For example, if $\mathsf{Or}(P, Q)$ could be matched to produce a `Bool`, the proofs $\mathsf{Inl}(t)$ and $\mathsf{Inr}(t)$ of `Or(True, True)` are equal yet would give `true` and `false`, so `true = false`. Large elimination is therefore allowed only from a *subsingleton*, a proposition with at most one possible proof: no constructors (`False`, where there is nothing to match) or one constructor whose fields are all proofs (`True`, `And`, an equation). Lean has the same rule.

Dependent fields. A field whose type mentions earlier fields makes its constructor *dependent*; the earlier fields it mentions are *index fields*. Writing, borrowing or moving a field of a dependent value *opens* it ([Open]): while open, a data field is typed by its content, assigning it is a strong update, and a write or borrow through an index field sets the proof fields whose type mentions it to $\bot$ (an error to read until reassigned). A value must be *whole* again (every dependent constructor inside it at its telescope type, no $\bot$ inside) wherever it passes to code that relies on its type ([Repack]); a part still lent at that point is checked as an unknown of its borrow's type, since the borrow may still write anything of that type there. Where a checked body returns, its result and every place it holds by borrow must be whole ([Return]); [Def] and closures check this at their body's return, and a stuck block at the end of each arm (it is an anonymous function, so its arms are its bodies).

2.7 Conversion

Conversion $v \equiv w$ is the *least* relation closed under: [Conv-refl] identity up to bound-variable names; [Conv-cong] same outermost former with pairwise convertible components (a constructor value's recorded parameters ignored;

sealed programs compared by their program text); [Conv-unit] $\mathsf{And}(\mathsf{True}, P) \equiv P \equiv \mathsf{And}(P, \mathsf{True})$; [Conv-pi] two Π-closures with the same class and borrow flag whose parameter types and codomains, evaluated at the generic arguments of the first, are pairwise convertible; [Conv-fun] two function values whose Π-types are convertible, whose captured values are pairwise convertible, and whose generic calls have convertible observations (with one shared σ_w for a borrow result). There is no η-rule; values at `Unit` are `()` wherever their type is known to be `Unit`, and every proof is $\star$.

2.8 Refinements, valuations and the two paths

Definition 1 (refinement). A *refinement* α assigns values to abstract values, $\sigma \mapsto v$. It *respects* a state Ω when each σ it refines is an abstract value of Ω and $\alpha(\sigma)$ is borrow-free and has a type convertible with $\Delta(\sigma)$. It is applied to a state, value or term by substituting and then restoring normal form: every sealed program that changed is re-run ([Seal]) and every `Eq` type rebuilt by `eq`. We write $\Omega\alpha$, $v\alpha$, $t\alpha$; α *covers* Ω when it assigns a value to every abstract value of Ω. More generally, $\Omega[w/a]^+$ replaces every occurrence of an abstract value, loan or sealed program a by w in the values of Ω, its stored types, Δ and the goal, and then restores normal form; $\mathrm{nf}(v)$ is the normal form of v. A refinement is *ground* (a *valuation*) when every value it assigns contains no abstract value, no sealed program and no inert loan.

Refinements cover the three ways the checker instantiates an unknown: a case split ($\sigma := \mathsf{C}(\overline{\sigma}')$), a call site (a parameter's generic σ instantiated with the argument), and an abstract function instantiated with a closure. Borrow-freeness costs nothing: a borrow parameter's generic value is $\mathrm{borrow}_\ell\sigma$ with σ borrow-free, and since `&` is never produced by computation and never an argument, no unknown type is ever refined to a borrow type.

The two paths. Every statement is evaluated along two paths: at a definition's generic call, where calls and matches on abstract values close off, and directly at each instance, where they run. [Call-type] and [Split] identify the results, so a proof checked on the first path is used on the second. If the paths take any decision differently, and the decision changes a result, a statement proved on one path can be false on the other: a closed proof of `False`. The design principle is that *no decision that changes a result is read from a normal form.*

Ground instances. A *ground instance* of a definition d is a valuation β of its generic σ's; its state is the generic state under β; its *hypotheses* are the proof parameters' types there, evaluated left to right; its *goal* is the codomain there, which is also the [Call-type] of the call $d(\overline{a}\beta)$ from a caller whose frame holds the cells.

3 The target

Definition 2 (accepted program). A *program* is a finite list of inductive declarations and definitions, read in order. It is *accepted* when each item is accepted by [Ind-decl], [Def] or [Const] against the signature of the items before it, the library's declarations (`Pair`, `False`, `True`, `And`, with `Nat` and `Unit` built in) coming first. It is *hole-free* when no term in it is a hole (`?`, also written `sorry`). Every definition has a body, so holes are the only axioms.

Target 3 (consistency of the calculus). Assume ZFC with one inaccessible cardinal per universe level used (both defined in Section 4.1.1). For every accepted, hole-free program, no definition without parameters has a type that evaluates to `False`. Equivalently, no closed term has type `False`, nor any type, such as `Eq(Nat, Z, S(Z))`, that computes to it.

The Lean statement against the checker is `Ochr.Consistency.consistency`:

```
theorem consistency (q : Program) (fuel : Nat)
    (holes : NoHoles q) (acc : Accepted (Prelude.decls ++ q) fuel) :
    ∀ d ∈ q, ¬ d.IsFalseConst
```

`Prelude.decls` is the library's declarations. `Accepted p fuel` says that the checker's entry point `checkProgram` gives every declaration of `p` the verdict *accepted* at that fuel (not *rejected*, and not *undecided*, which is what running out of fuel or a depth limit gives). `IsFalseConst` says a declaration is a definition with no parameters whose declared type is the identifier `False`.

How the two statements relate.

- **Rules and checker.** Target 3 is about the rules; the Lean statement is about the checker that implements them. Every step of this plan is phrased so that it can be stated against the checker's functions, so a completed plan would prove both, the Lean form being the stronger since it also excludes implementation errors.
- **A literal `False` loses nothing.** If an accepted program had a parameterless c whose type merely computes to an empty type, one more fresh declaration turns it into a literal one: `def B : False := c` when the type computes to `False`; a one-arm match for `False ∧ ⊤`; an annotated zero-arm match for a user's empty inductive. The Lean statement quantifies over all programs, so it covers these.
- **Fuel.** Exhaustion is an outcome of its own that no rule below the verdict catches, and a caught failure cannot refund fuel, so a decided verdict is the same at every larger fuel. The Lean statement quantifies over every fuel and does not need this fact, but a proof may use it to fix a fuel large enough for a given run.
- **Totality.** Every function the checker runs is total, with explicit fuel where recursion is not structural, so the kernel can unfold every step. The one exception, an opaque address used to key the editor's location table, is never consulted when locations are off, which `Accepted` ensures.
- **Universes.** Lean's strength is ZFC with n inaccessibles for every $n < \omega$. If the set model for Ochr needs one inaccessible per level at once, the uniform Lean statement is unprovable in Lean (Gödel's second incompleteness theorem: a theory cannot prove the consistency of one at least as strong, Section 4.1.1), and must be weakened to bounded universe levels, made relative to an assumed model, or stated per program (Section 10, Q8).

> **Quick explainer: Lean's kernel and its axioms.** Lean's *kernel* is the small trusted core that checks every proof; tactics and the elaborator only produce proof terms for it to check, so a Lean theorem is believed exactly as far as the kernel is. Beyond its core type theory, Lean has three standard axioms: `propext` (two propositions that imply each other are equal), `Quot.sound` (elements related by a relation are equal in its quotient) and `Classical.choice` (from a proof that something exists, pick one). All three hold in Lean's set model. `#print axioms t` lists the axioms a theorem t depends on; an unfinished proof shows up there as `sorryAx`. A `partial def` is opaque to the kernel: it can run, but a proof cannot unfold it, which is why a provable statement about the checker needs every function the checker runs to be total.

4 The shape of the argument

The argument has three parts, as in the paper's proof for its fragment: a model that says what "true" means; a soundness theorem saying the checker accepts only what is true; and consistency as a corollary.

4.1 The set model

4.1.1 Background: sets, universes and models

This subsection defines everything the model uses. It follows the standard set model of Lean (Carneiro, *The Type Theory of Lean*, a copy of which is at `docs/papers/theory-of-lean.pdf`), but nothing below depends on having read it. For readers who want the full construction for Lean, of which Section 4.1 is an adaptation, the relevant parts are, optionally: §1.2 (why sets model a type theory, the two-element `Prop`, and why inaccessible cardinals are needed); §6.2 (the interpretation map, with universes $U_{n+1} = V_{\kappa_n}$ and the clauses for Π, `∀` and `=`); §6.2.1 (the least-fixed-point construction of inductive types in ZFC); and §6.3, Theorem 6.7 (the soundness of the model: types denote sets, proofs denote $\bullet$, convertible terms denote the same thing).

ZFC, and consistency relative to it. ZFC is Zermelo–Fraenkel set theory with the axiom of choice, the usual foundation of mathematics. A theory T is *consistent* if it does not prove a contradiction. By Gödel's second incompleteness theorem, a consistent theory cannot prove its own consistency, nor that of a theory at least as strong; so a consistency proof for a strong type theory is always *relative*: "if ZFC plus some axioms is consistent, so is the type theory".

Quick explainer: countable and uncountable sizes. Two sets have the *same size* if there is a bijection between them (a pairing that matches every element of one with exactly one element of the other); $|A|$ is the size of A, and $|A| \le |B|$ means A can be matched one-to-one into part of B. A set is *countable* if it is finite or has the same size as the natural numbers $\mathbb{N}$, that is, its elements can be listed $a_0, a_1, a_2, \ldots$; the integers, the rationals and the set of all finite strings are countable. A set is *uncountable* if it is not. Cantor's theorem says that no set A has the same size as $\mathcal{P}(A)$, the set of all its subsets: $\mathcal{P}(A)$ is always strictly bigger. So $\mathcal{P}(\mathbb{N})$, equivalently the real numbers or the set of all functions $\mathbb{N} \to \mathbb{N}$, is uncountable, and the sizes go on growing forever: $\mathbb{N}$, $\mathcal{P}(\mathbb{N})$, $\mathcal{P}(\mathcal{P}(\mathbb{N}))$, …

Quick explainer: cardinals. A *cardinal* is a size. The size of $\mathbb{N}$ is written $\aleph_0$ ("aleph-nought"), the smallest infinite cardinal; the cardinals bigger than it are the *uncountable* cardinals. If $\lambda = |A|$, then 2^λ is the size of $\mathcal{P}(A)$ (a subset of A is a choice of in or out for each element, hence the notation); by Cantor, $2^\lambda > \lambda$ always. Cardinals are ordered by $\le$ above, and "fewer than κ sets" means a collection of them whose size is less than κ.

Quick explainer: ordinals. Ordinals extend counting past the natural numbers. They measure *position* in a sequence, not size: $0, 1, 2, \ldots, \omega, \omega+1, \omega+2, \ldots, \omega\cdot 2, \ldots$, where ω is the first position after every natural number. Every ordinal other than 0 is either a *successor*, which has an immediate predecessor ($3 = 2+1$; $\omega+1$ comes straight after ω), or a *limit*, which has none and is reached only as the limit of all smaller ordinals (ω, $\omega\cdot 2$, …). Different ordinals can have the same size: ω, $\omega+1$ and $\omega\cdot 2$ are different positions but all countable. Each cardinal is identified with the first ordinal of that size, so $\aleph_0$ is ω, and every infinite cardinal is a limit ordinal, though most limit ordinals are not cardinals. This identification is what lets a cardinal κ index a stage V_κ below.

Definition 4 (cumulative hierarchy). $V_0 = \emptyset$; $V_{\alpha+1} = \mathcal{P}(V_\alpha)$, the set of all subsets of V_α; and $V_\lambda = \bigcup_{\alpha<\lambda} V_\alpha$ for a limit ordinal λ. Every set belongs to some V_α.

Definition 5 (inaccessible cardinal). An uncountable cardinal κ is *inaccessible* if it is *regular* (a union of fewer than κ sets, each of size less than κ, has size less than κ) and a *strong limit* ($2^\lambda < \kappa$ whenever $\lambda < \kappa$).

The two conditions say that κ cannot be reached from below by the two ways of making sets bigger: taking a union of fewer than κ smaller sets, and taking all subsets of a smaller set. $\aleph_0$ satisfies both (a finite union of finite sets is finite, and a finite set has finitely many subsets), which is why the definition asks for an uncountable κ: an inaccessible is to the uncountable cardinals below it what $\aleph_0$ is to the finite numbers.

If κ is inaccessible, V_κ is closed under every construction a type theory needs: for $A \in V_\kappa$ and a family $B(x) \in V_\kappa$ indexed by $x \in A$, the set of subsets of A, the *dependent sum* $\Sigma(x \in A)B(x)$ (the set of pairs (x, y) with $x \in A$ and $y \in B(x)$; when A is a set of tags this is a *disjoint union*, written $\biguplus_{x\in A} B(x)$) and the *dependent product* $\prod_{x\in A} B(x)$ (defined below) are again in V_κ. So V_κ can serve as "the type of all small types". ZFC cannot prove that an inaccessible exists (with one, V_κ would be a model of ZFC, proving ZFC consistent), so the existence of inaccessibles is the extra axiom the target assumes.

Notation. Throughout, $[\![T]\!]$ is the set a type T is interpreted as, and $[\![v]\!]$ the element a value v is interpreted as.

Universes. Fix inaccessibles $\kappa_0 < \kappa_1 < \ldots$, one for each universe level used. Type_i is interpreted as V_{κ_i}: a type in Type_i is a set in V_{κ_i}. Since $V_{\kappa_i} \in V_{\kappa_{i+1}}$, the rule $\mathsf{Type}_i : \mathsf{Type}_{i+1}$ holds in the model.

Propositions, proofs and proof irrelevance. Fix an object $\bullet$, "the proof". A proposition is interpreted as $\emptyset$ (false) or $\{\bullet\}$ (true), so `Prop` is interpreted as the two-element set $\{\emptyset, \{\bullet\}\}$, and every proof is interpreted as $\bullet$. Any two proofs of one proposition therefore have the same meaning; this *proof irrelevance* is what makes it sound for the calculus to represent every proof by the single value $\star$. A *subsingleton* is a set with at most one element; every proposition denotes one.

Dependent products. A function type $\Pi(x : A).B$ whose codomain is data or a type is interpreted as the dependent product $\prod_{x\in[\![A]\!]} [\![B]\!](x)$: the set of functions f with domain $[\![A]\!]$ such that $f(x) \in [\![B]\!](x)$ for every x.

Products of propositions, and impredicativity. When B is a proposition the product cannot be used as it stands, because it is not a truth value, but it is always one step away from one. Each $[\![B]\!](x)$ is $\emptyset$ or $\{\bullet\}$, so there are two cases. If $[\![B]\!](x_0) = \emptyset$ for some x_0, no function can choose a value at x_0, and the product is $\emptyset$. If $[\![B]\!](x) = \{\bullet\}$ for every x, each $f(x)$ is forced to be $\bullet$, so exactly one function qualifies, the constant function

$x \mapsto \bullet$ (when $[\![A]\!]$ is empty, it is the empty function), and the product is $\{x \mapsto \bullet\}$. So the product always has at most one element: it is empty exactly when "$[\![B]\!](x)$ is true for every x" fails. But its one element, when there is one, is a function, not $\bullet$, so the product is a singleton without being the set $\{\bullet\}$. The model therefore interprets a Π into a proposition not as the product but as the truth value the product carries:

$$[\![\Pi(x : A).B]\!] = \{\bullet \mid [\![B]\!](x) = \{\bullet\} \text{ for every } x \in [\![A]\!]\},$$

which is $\{\bullet\}$ when the product is a singleton and $\emptyset$ when it is empty. This collapse loses nothing, and it is forced by proof irrelevance: every proof denotes $\bullet$, so a proof of $\Pi(x : A).B$, for instance a `λ`, must denote $\bullet$ too rather than the function $x \mapsto \bullet$, and applying such a proof to an argument denotes $\bullet$, which is what $[\![B]\!](x)$'s one element is. Since no rule can look inside a proof, nothing can tell the collapsed meaning from the product it replaces. The result is always $\emptyset$ or $\{\bullet\}$, a member of `Prop`'s interpretation $\{\emptyset, \{\bullet\}\}$, *however large* $[\![A]\!]$ is, even when A is `Prop` itself or a universe. This is *impredicativity*: a proposition may quantify over all propositions, or over all types of a universe, without moving to a larger universe. The set model gets it for free because the size of the domain never reaches the result. The same move is unavailable for data: `Π(A : Type₀). A → A` is a set of functions on all of V_{κ_0}, which is too large to be a member of V_{κ_0} again, so a Π over a universe into data must live in a larger universe.

Least fixed points. An operator Φ on families of sets is *monotone* if $X \subseteq Y$ (pointwise) implies $\Phi(X) \subseteq \Phi(Y)$. Its *least fixed point* is the smallest family X with $\Phi(X) \subseteq X$, the intersection of all such families, which exists for a monotone operator acting inside a fixed set such as V_κ. An inductive type is interpreted as the least fixed point of the operator "one more layer of constructors". In Ochr no recursive occurrence of a type lies under a Π (fields are first-order), so every element is built by finitely many constructor applications, and the least fixed point is the union $\bigcup_{n<\omega} \Phi^n(\emptyset)$, which stays inside V_κ.

Why a model gives consistency. A model assigns to each type T a set $[\![T]\!]$ and to each well-typed term $t : T$ an element $[\![t]\!] \in [\![T]\!]$; that it does so for every derivable typing is the model's *soundness*. Since $[\![\mathsf{False}]\!] = \emptyset$, no closed term of type `False` can exist, and since the model is built in ZFC with the inaccessibles, the conclusion is relative to that theory. For Ochr, "well-typed term" is replaced by "definition accepted by the checker", and the model's soundness is the soundness theorem of Section 4.3, which relates the checker's runs to meanings.

4.1.2 The interpretation

Types are interpreted as sets in the way just described, adapting Werner's model of the calculus of inductive constructions (Coq's type theory) and Carneiro's of Lean. As above, $[\![T]\!]$ is the meaning of a closed type value T, and $[\![v]\!]$ of a closed value.

Definition 6 (meaning of types).

- $[\![\mathsf{Type}_i]\!] = V_{\kappa_i}$ and $[\![\mathsf{Prop}]\!] = \{\emptyset, \{\bullet\}\}$.
- An inductive declaration in $\mathtt{Type}_0$ denotes, simultaneously for every value of its parameters, the least family of sets closed under its constructors: the least fixed point of the operator sending a family X to $\bar{a} \mapsto \biguplus_i \{\mathsf{C}_i\} \times \Sigma(g_1 : B_1)...\Sigma(g_k : B_k)$, where each field type is read at the parameters $\bar{a}$ and the earlier fields, with every occurrence $\mathsf{D}(\overline{b})$ read as $X(\overline{b})$. Non-uniform occurrences make the fixed point simultaneous over the whole family; it is still the least fixed point of a monotone operator on families.

  For example, `inductive List (A : Type) := Nil | Cons(h : A, t : List(A))` has, at a parameter value T, the operator

  $$\Phi(X) = \{\mathsf{Nil}\} \uplus (\{\mathsf{Cons}\} \times [\![T]\!] \times X),$$

  writing $\mathsf{Cons}(a, l)$ for the triple (Cons, a, l). Its iterates from $\emptyset$ are $\Phi^0(\emptyset) = \emptyset$; $\Phi^1(\emptyset) = \{\mathsf{Nil}\}$, since `Nil` has no fields and a `Cons` needs an element of $X = \emptyset$; $\Phi^2(\emptyset)$ adds the one-element lists $\mathsf{Cons}(a, \mathsf{Nil})$ for $a \in [\![T]\!]$; and in general $\Phi^{n+1}(\emptyset)$ is exactly the lists of length at most n, each stage containing the one before (by induction, since Φ is monotone). Their union $L = \bigcup_{n<\omega} \Phi^n(\emptyset)$ is therefore exactly the finite lists of elements of $[\![T]\!]$: a list of length n appears at stage $n+1$, and every stage holds only finite lists. L is a fixed point. An element of $\Phi(L)$ is `Nil`, which is in L, or $\mathsf{Cons}(a, l)$ with $l \in L$, so l is in some stage $\Phi^n(\emptyset)$ and $\mathsf{Cons}(a, l)$ in the next. Conversely an element of L lies in some $\Phi^{n+1}(\emptyset) = \Phi(\Phi^n(\emptyset)) \subseteq \Phi(L)$, by monotonicity. And L is the least such set: if $\Phi(Y) \subseteq Y$, as for any fixed point Y, then every stage is inside Y, by induction on the stage, since $\emptyset \subseteq Y$, and $\Phi^n(\emptyset) \subseteq Y$ gives $\Phi^{n+1}(\emptyset) \subseteq \Phi(Y) \subseteq Y$. So $L \subseteq Y$, and $[\![\mathsf{List}(T)]\!] = L$.
- An inductive declaration in `Prop` denotes the same fixed point, truncated: $\{\bullet\}$ if inhabited, $\emptyset$ otherwise. So $[\![\mathsf{False}]\!] = \emptyset$, $[\![\mathsf{True}]\!] = \{\bullet\}$ and $[\![\mathsf{And}(P, Q)]\!] = [\![P]\!] \cap [\![Q]\!]$.
- $[\![\mathsf{Eq}(A, a, b)]\!] = \{\bullet\}$ if $[\![a]\!] = [\![b]\!]$, and $\emptyset$ otherwise. An equation field denotes the same.
- A function type with borrow parameters $x_i : \&T_i$ for $i \in I$ denotes a subset R (defined next, Definition 7) of $\Pi(\overline{d} : \overline{D}).\mathrm{Out}_B(\overline{d})$, where $D_i = [\![T_i]\!]$ for $i \in I$ and $D_j = [\![A_j]\!]$ otherwise, $\mathrm{Fin} = \prod_{i \in I} [\![T_i]\!]$ is the tuple of final contents of the borrowed places, and $\mathrm{Out}_B(\overline{d})$ is $[\![B]\!](\overline{d})$ if B is a proposition, $[\![B]\!](\overline{d}) \times \mathrm{Fin}$ if B is borrow-free data, and $[\![T]\!] \times ([\![T]\!] \to \mathrm{Fin})$ if $B = \&T$. For F in this set, fwd_F and back_F are the two components of $F(\overline{d})$; $\mathrm{back}_F(\overline{d})$ is the *backward function*, mapping the value the caller leaves in the returned borrow to the final contents. A Π-type whose class is "returns proofs" is the exception: as argued in Section 4.1.1, it denotes not this set but the truth value it carries, $\{\bullet\}$ if $[\![B]\!](\overline{d}) = \{\bullet\}$ for every $\overline{d}$ and $\emptyset$ otherwise (impredicativity). Its borrowed places need no final contents, since a proof's run is discarded.

  Example, a function that takes a borrow. In-place addition, `def AddM (x : &Nat) (y : Nat) : Unit`, which walks down `*x` to its final `Z` and replaces it with `y`. Its one borrow parameter is `x`, so $I = \{x\}$, both domains are $[\![\mathsf{Nat}]\!] = \mathbb{N}$, and $\mathrm{Fin} = \mathbb{N}$ is the final content of `*x`. The codomain `Unit` is borrow-free data, so a meaning is a function $\mathbb{N} \times \mathbb{N} \to [\![\mathsf{Unit}]\!] \times \mathbb{N}$, and `AddM` denotes $(n, m) \mapsto ((), n + m)$: started with `*x` holding n, it returns `()` and leaves $n + m$ in `*x`. The borrow is thus modelled as passing the place's starting content in and getting its final content back out.

  Example, a function that returns a borrow. `def TailM (x : &Nat) : &Nat`, which returns a borrow of the final `Z` inside `*x`. Again $I = \{x\}$ and $\mathrm{Fin} = \mathbb{N}$, but now $B = \&\mathsf{Nat}$, so a meaning sends the starting content n of `*x` to a pair in $\mathbb{N} \times (\mathbb{N} \to \mathbb{N})$, and `TailM` denotes $n \mapsto (0, v \mapsto n + v)$. The first component, $\mathrm{fwd}(n) = 0$, is what the returned borrow holds when the call returns: the final `Z`. The second, $\mathrm{back}(n) = (v \mapsto n + v)$, is the backward function: if the caller leaves v in the returned borrow before it ends, `*x` finally holds n with its final `Z` replaced by v, that is $n + v$. This backward function is injective (different v give different final contents), which is what the injectivity relation defined next demands of every function returning a borrow.
- $[\![\mathsf{Id}(A, t, u)]\!] = \{\bullet\}$ iff the meanings of the two observations of Section 2.4 are equal.

What R is for. For a function that returns a borrow, the backward function says how the final contents of the borrowed places depend on the value the caller leaves in the returned borrow. The plain product of functions contains every pair $(\mathrm{fwd}, \mathrm{back})$, including ones whose backward function ignores its argument entirely. The model must leave those out, because no Ochr function behaves that way and the checker accepts proofs that rely on it. R is the condition that leaves them out: the backward function of every function returning a borrow, at every type, must be *injective*, so that different values left in the returned borrow give different final contents.

Why every Ochr function has an injective backward function. A function type returning a borrow must have a borrow parameter, so Ochr has no `'static` borrows: a returned borrow is always derived from one of the function's borrow arguments, and points at some position inside the places they lend. When it ends, the value the caller left in it is put back at that position. So the final contents still hold that value at that position, and two different values give two different final contents. For `TailM` above, $\text{back}(n) = (v \mapsto n + v)$: the value left in the borrow becomes the final `Z`'s replacement, and can be read back off the result. A function returning a leaked borrow would be different: Rust allows a `fn(&mut u64) -> &mut u64` to return `Box::leak(Box::new(0))`, a borrow of a fresh place that nobody owns, whose backward function is constant because what is written through it never reaches the caller's places.

Why the model must build it in. The checker relies on it, so a model without it would make an accepted program false. Take

```
def PF (x : &Nat) (e : Id(Unit, *x := 0, *x := 1)) : False := e
```

At its generic call the statement observes the owner of `x`, where the two sides leave `0` and `1`, so it computes to $\mathsf{Eq}(\mathsf{Nat}, 0, 1)$, which is `False`, and `PF` is accepted: writing `0` and writing `1` through a borrow can never be indistinguishable. Now call `PF` with a borrow returned by an unknown function $g : \Pi(x : \&\mathsf{Nat}).\&\mathsf{Nat}$. Its hypothesis says that writing `0` and writing `1` through $g(x)$ leave the same contents, which in the model says $\text{back}_g(n)(0) = \text{back}_g(n)(1)$. If the meaning of that function type contained a g with a constant backward function, this hypothesis would be true for that g, and `PF` would turn it into a proof of `False` that is true in the model, which no sound model allows. So the meaning of a function type returning a borrow must contain only functions with injective backward functions, which is R.

Definition 7 (injectivity relation). For each type A, $R_A \subseteq [\![A]\!]$ is the set of values of A all of whose borrow-returning functions, however deeply nested, have injective backward functions. It is defined by induction on A. For propositions and universes $R_A = [\![A]\!]$. At an inductive type, R holds of a constructor value when it holds of each field. $R_{\&T} = R_T$. For a function type, $F \in R_{\Pi(\overline{x:A}).B}$ iff for all arguments in R, $\text{fwd}_F\left(\overline{d}\right) \in R_B$ and, if $B = \&T$, $\text{back}_F\left(\overline{d}\right)$ is injective.

Open design question Q9 (a well-founded definition of the model). As written, the interpretation (Definition 6) and R (Definition 7) are not yet a valid definition, for two reasons.

- *They are circular at function types.* Definition 6 makes the meaning of a function type returning a borrow equal to the subset R of a product, while Definition 7 defines R_A as a subset of the meaning of A. The intended order is: the plain product first, then R as a condition on it, then the meaning as the part of the product satisfying R.
- *"By induction on A" has nothing smaller to recurse on.* A function type's codomain is computed by running code at each argument (a type function such as `Cells(E, n)` is defined by recursion on data), not a subterm of A; and a type variable, such as a parameter $A : \mathsf{Type}_0$, has no structure at all, so R is undefined there.

The likely repair is to define the two together, interpreting each type as a set *paired with* its subset of good elements, and each universe as a set of such pairs, so that a type variable denotes a pair and R at it is the pair's second component; the recursion through computed types then rests on a denotation of type-level code. That denotation is the same object the gap between truth in the model and the soundness walk over machine values needs (Section 9), so this question should be settled together with that one. This is a matter of stating the model precisely, not of whether it exists.

Definition 8 (truth). A closed proposition value P is *true*, $\vDash P$, when $[\![P]\!] = \{\bullet\}$.

In the paper's fragment, truth was defined syntactically on ground proposition values (`True`, `False`, `And` and stuck equations, the last counting as true). Beyond the fragment, propositions include Π-types (implication and universal quantification over data, functions and propositions), user inductives in `Prop`, and `Eq` at any type, so truth must be semantic. Whether a stuck equation still counts as true is Q3 (Section 10).

4.1.3 Conditions under which the model is defined

Lemma 9 (the model is well defined). If every inductive declaration satisfies (I1)–(I6) below, every match on a proof satisfies (I7), recursion satisfies (I8), `Eq` satisfies (I9), and borrows are only of `Type₀`, then Definition 6 assigns a set to every type value formed by an accepted program, in the universe of its sort.

- (I1) *Strict positivity.* The declared type `D` occurs in a field type only as the whole field type, or inside another inductive at a *nestable* parameter (one occurring in that inductive's fields only as a whole field type or at a nestable parameter); never left of a Π, under `&`, or as the argument of a type function.
- (I2) The arguments of an occurrence of `D` never mention `D`.
- (I3) A parameter is never called in a field type; only earlier global type functions are.
- (I4) Parameter types do not mention `D`.
- (I5) Fields form a telescope: a field's type mentions only parameters and earlier fields.
- (I6) Every field of an inductive in `Type₀` or `Prop` lives in `Prop` or `Type₀`.
- (I7) A match on a proof produces data or a type only from a subsingleton (no constructors, or one whose fields are all proofs).
- (I8) Recursion is never on a proof.
- (I9) `Eq` decides only by conversion (`True`) or by distinct constructors of one inductive (`False`); otherwise it is stuck. So an equation at any type, functions and sorts included, adds no rule that could prove `False`.

Proof sketch. (I1)–(I5) make the constructor operator monotone on families, so the least fixed point exists; (I6) keeps it in the sort's universe; (I7) is what makes a match on a proof well defined, since all proofs are • and only a subsingleton's eliminator cannot tell them apart; (I8) because a proof has no structure to measure; (I9) because constructors are injective with disjoint images. Borrows only of `Type₀`: borrowing a universe would put `Π(x : &Type₀)(a : *x). *x` in `Type₀`, which would be impredicative and have no set model. □

> **Quick explainer: strict positivity and W-types.** An inductive declaration is read as "the least set closed under its constructors", and such a set exists only if the type being declared is used *positively* in its fields. A field that is a function *out of* the type breaks this, and no set fits the declaration. In Ochr, with `inductive Bad := MkBad(f : Π(x : Bad). Empty)`:
>
> ```
> def L (b : Bad) : Empty := match b { MkBad(f) => f(b) }
> def bad : Bad := MkBad(λ(x : Bad) : Empty => L(x))
> def Boom : False := absurd(L(bad))
> ```
>
> `L` applies `b`'s own field to `b` itself; `L(bad)` would loop forever, but `Boom` is typed by its callee's statement alone and proofs are not run, so it would be a closed proof of `False`. (Before the positivity rule existed, the checker accepted exactly this.) Linearity does not stop it: `f` is used once, and `b` is consumed only once, by the call, because the match does not consume it (it names `f` as a sub-place of `b`). A language where a match consumes its scrutinee, such as Bend, must rebuild the value and write `f(MkBad(f))`, using `f` twice, which affinity rejects; Ochr's matches do not consume, so it needs positivity instead. *Strict positivity* is the syntactic rule that rules this out: the type being declared appears only as a whole field type, or nested inside another inductive that itself uses it positively, never left of an arrow. A *W-type* is an inductive with a field that is a function *into* the type, such as `Tree := Node(children : Π(n : Nat). Tree)`, an infinitely branching tree. That is strictly positive and Lean allows it, but its least fixed point is reached only after transfinitely many stages of the construction, which the model must have room for. Ochr forbids any function-typed field, so it has no W-types, and its fixed points are reached after ω stages (Section 4.1.1). This is one reason Ochr may be weaker than Lean (Q8).

Status. paper argument. Every condition is enforced by [Ind-decl], the match rules and [Rec]; a probe of each condition against the checker found none missing. One condition is enforced more strictly than (I9) needs: equation fields are required to be over first-order data, which a type parameter instantiated at a function type bypasses harmlessly.

4.2 Conversion is sound

Lemma 10 (conversion respects meaning). If v and w are closed values (each with no free variables and no unknowns, Section 2) and $v \equiv w$ (definitionally equal), then $[\![v]\!] = [\![w]\!]$. Consequently, at a ground state, convertible propositions are equally true.

The argument is one principle per conversion rule:

- (V1) Normal forms are compared structurally; the meaning of a normal form is a function of its parts.
- (V2) A constructor value means its tag and fields; the parameters it records are fixed by its type and never read, so ignoring them is sound.
- (V3) A sealed program is a closed program, so two with the same text have the same meaning, and refinement re-normalises both alike.
- (V4) A Π-type is fixed by its parameter types and codomain at its generic arguments, by (P), the lemma that a run cannot depend on an unknown it never inspects (Lemma 31), and its class and borrow flag are part of its meaning: they decide which functions inhabit it and how their calls close off.
- (V5) A function value is fixed by the observation of its generic call: result, what a returned borrow holds at return, the final contents after one fresh σ_w is written through it, and the final contents of the generic owned places. This is complete because a call's only effects are its result and its borrowed places (the frame property, Lemma 35), a backward function only ever puts the written value where the borrow points (holes are never inspected), a call that inspects a generic argument closes off there (so its observation is its own sealed call, convertible only with itself), and a call that does not behaves uniformly at every instance by (P).
- (V6) Conversion is least: a comparison that needs itself answers "not convertible". Read coinductively, any two functions stuck at their generic calls would be identified, and `J` along that proves `Eq(Nat, S(Z), Z)`.
- (V7) The unit laws and η at `Unit` are equalities of sets: $[\![\mathsf{True} \wedge P]\!] = [\![P]\!]$, and `Unit` has one element.
- (V8) `Eq` computes by meanings: reflexivity by (V1)–(V7), injectivity and disjointness by (V2), and an equation over $\bot$ stays stuck. `J` returns its argument only at convertible endpoints, where it is the identity on meanings.
- (V9) Conversion is monotone under refinement in the safe direction: refinement maps normal forms to normal forms functionally, so convertible values stay convertible (the function case by (P)); in the other direction, every consumer of "not convertible" either fails, stays stuck, or re-checks later, so no consumer is more permissive on "not convertible" than on "convertible".

Status. paper argument, resting on (P) for (V4), (V5) and (V9). Every rule is enforced by the checker's conversion; 35 probes, among them transport along true equations between in-place programs, behave as predicted.

4.3 The soundness theorem

The paper proves the following for a first-order fragment F (Section 6.1), conditional on two assumptions restated in Section 6.2. The plan is to prove it for the whole calculus.

Target 11 (soundness). Let $\mathcal{P} = d_1 ... d_m$ be an accepted, hole-free program. For every d_k and every ground instance β of d_k whose values are *good* (a notion still to be fixed; Q1 below): (i) if d_k is a *data function* (its codomain is data or a borrow type), its body runs from the ground instance to completion without error, in runtime mode and in erased mode, and its frame's pop succeeds; if it returns a borrow, the result is live, not $\bot$; (ii) if d_k is a *lemma* (its codomain is a proposition) and its hypotheses at β are true, its goal at β is true; (iii) every function value that a run of (i) or (ii) produces is good.

Clause (iii) is new beyond F, where there are no function values. What "good" means for a ground function value is the first open design question.

> **Quick explainer: logical relations.** A *logical relation* defines, for each type, which values "behave like" that type, by induction on the type: a value is good at `Nat` if it is a numeral; a function is good at $\Pi(x : A).B$ if it sends every good argument to a good result; nothing is good at `False`. One then proves the *fundamental lemma*, that every program the checker accepts is good at its type, one typing rule at a time; consistency follows because nothing is good at `False`. It is called "logical" because each clause follows the type's logical shape (good inputs to good outputs). Impredicativity is the difficulty: the clause for $\Pi(P : \mathsf{Prop}).B$ needs "good at P" for every proposition P, including that Π-type itself, so the definition is no longer by induction on anything smaller. Girard's *reducibility candidates* fix this by quantifying over every

set of values that could be a proposition's meaning, instead of over the propositions themselves; the technique is standard but technical. A first worked example, for a tiny language, is the chapter "Norm" of *Software Foundations*, volume 2.

Open design question Q1 (good ground values). In F every ground value is a numeral or `()`. Beyond F a ground instance can bind a parameter to a closure, and a hypothesis can be a Π-type over functions or propositions. Target 11 needs a notion of *good* ground value that is (a) closed under what accepted code produces, (b) strong enough that calling a good closure at good arguments terminates without error and returns a good value (so (i) holds for calls through function parameters, whose bodies are not earlier definitions and are not covered by the induction), and (c) such that a good function value's meaning lies in R (Definition 7). Two candidates:

- *A logical relation*, defined by induction on types: a closure is good at $\Pi(x : A).B$ when it maps good arguments to good results. Ochr's `Prop` is impredicative, so a type like `Π(P : Prop). P → P` quantifies over all propositions, and a definition by induction on types breaks down there; it would need reducibility candidates in Girard's style, interpreted in the set model.
- *Definability*: quantify only over ground instances whose function values are closures formed by accepted code, and strengthen the induction to cover closures as anonymous definitions, ordered by where they were formed. Consistency only needs closed terms, whose instances are built this way, so this may suffice; it needs a measure under which a closure formed in d_k and called from d_j is smaller.

Until one is chosen, every statement below that quantifies over function values (the soundness theorem's "good ground instances", agreement clause (A6), the walk's step for a call through a function value, and the recursion lemma) is written relative to an unspecified notion of "good", and is incomplete. The features beyond F that involve no function values (dependent fields, Fording, stuck blocks and `J` over data) do not depend on this choice.

Corollary 12 (consistency, from soundness). Under Target 11 and Lemma 9, Target 3 holds.

Proof sketch. Let $c : A$ be an accepted parameterless definition. It is checked from the empty state, which is ground, by [Const]; Target 11 (ii) with the empty instance gives $\vDash T$ for the value T of A. By Lemma 10, a type convertible with `False` is false, since $[\![\mathsf{False}]\!] = \emptyset$. So $T \not\equiv \mathsf{False}$. □

Corollary 13 (adequacy of `Id`). Under Target 11, if a lemma with codomain $\mathsf{Id}(D, t, u)$ is accepted, then at every good ground instance whose hypotheses are true, the observations of t and u have equal meanings: they return equal results and leave equal contents in every place either may write, except places one of them leaves moved out.

4.4 How soundness is proved: the walk

Reminder.

- $\mathcal{P} = d_1 ... d_m$ is the accepted program, and d_k is its k-th definition, the one being proved; the induction may assume the theorem for every earlier d_j, $j < k$ (Target 11).
- d_k is checked at its *generic call*, where each parameter is bound to a fresh abstract value σ ([Def], Section 2.6). If d_k is declared **by** x_j, σ_j is the abstract value bound to that decreasing parameter.
- β is a *ground instance* of d_k: a valuation, giving each of those σ's a value with no unknowns in it (Section 2.8). So $\beta(\sigma_j)$ is the actual value of the decreasing parameter at that instance, and $|\beta(\sigma_j)|$ is its size.
- α is a *refinement*, an assignment of values to abstract values (Definition 1); $\alpha \supseteq \beta$ means α agrees with β and also assigns the abstract values the check creates later, such as a split's fields.
- d_k's *case tree* is the tree its check builds: a node for each [Split] or [Split-gen] on an unknown, with one child per arm, and a leaf where a tail term ends ([Tail-end], Section 2.6).

The idea. The checker never ran a definition on real inputs: it checked each one once, at its generic call, on unknowns. To show that what it accepted is true, take any real instance β, run the definition on it for real, and walk the checker's case tree in step with that run. The tree is a finite record of every case the checker considered; the real run takes exactly one path through it, the one β selects. At every step the symbolic state and the real

state are kept related, so whatever the checker concluded about the first holds of the second. At the leaf the real run reaches, the checker had established the goal for that branch, so the goal holds at β.

The proof is by induction on the pairs $(k, |\beta(\sigma_j)|)$, ordered lexicographically, where $|v|$ is the size of the value in d_k's decreasing position (read through the borrow for &D; 0 without `by`).

Why: a real run of d_k makes calls, and each must already be known to be safe, and, for a lemma, true. There are only two kinds. A call to an earlier definition d_j, $j < k$, is covered by the first component: the theorem is already proved for everything earlier in the program, and a definition can only call earlier ones, so there are no cycles. A recursive call of d_k itself is covered by the second: [Rec] checked that its decreasing argument is a strict part of the entry value, so it is the same theorem at a smaller input. Ordering the pairs lexicographically says exactly "assume the theorem for anything earlier in the program, and for this definition at any smaller input". It is the induction that follows the program's own call structure, and the same measure that makes accepted recursion terminate. Without `by`, a definition never calls itself, so the size plays no part.

For fixed k and β, it follows d_k's case tree along β and maintains five things:

- a node of the tree with its symbolic state Ω_s;
- a refinement $\alpha \supseteq \beta$ that covers and respects Ω_s;
- the ground state Ω_g the ground run of the body has reached;
- *agreement* $\Omega_s \approx_\alpha \Omega_g$, the simulation relation between the two states (Definition 17);
- for a lemma, the invariant (Inv): every *proof place* of Ω_s (a binding declared a proof, or a field place of one) has a type T with $\vDash T\alpha$.

Why: the node and Ω_s say where the checker is; Ω_g says where the real run is; α is the dictionary from the checker's unknowns to the real run's values, and it grows as the walk learns more (a split tells it what a value's fields are); agreement is the promise that the two states describe the same thing once translated by α; and (Inv) says every hypothesis the checker relied on is actually true at the instance, which is what makes using a proof sound.

At the root $\Omega_s\beta = \Omega_g$ (nothing has run yet, and the ground instance's state is by definition the generic state under β), so agreement holds, and (Inv) holds because each hypothesis's type at the generic state is true under β iff it is true at the ground instance (Lemma 21). The tail rules move down the tree:

- **[Tail-let], [Tail-seq].** The term run is not a match on data in tail position; Lemma 23 gives a successful ground run and keeps agreement.

  Why: these steps do not branch on an unknown, so both runs do the same thing; all the difficulty is inside Lemma 23, the lemma that the two paths agree.

- **[Tail-match].** After [Access], agreement makes the scrutinee's head constructor the same on both paths, so both take the same arm.

  Why: when the value matched on is already known to the checker, there is nothing to decide: both runs see the same constructor.

- **[Tail-split] on σ.** The ground scrutinee's head is that of $\alpha(\sigma)$, say C_i; move to arm i and extend α to the arm's fresh field values by the fields of $\alpha(\sigma)$. α respects $\sigma := \mathsf{C}_i(\overline{\sigma})$ and $(\Omega_s[\sigma := \mathsf{C}_i(\overline{\sigma})])\alpha = \Omega_s\alpha$, so agreement and (Inv) carry over.

  Why: this is where the checker considered every case and the real run takes one. The walk follows the arm the real value selects, and α learns the values of its fields. The checker proved every arm, so in particular the one taken.

- **[Tail-gen] on a sealed program n.** Extend α by $\alpha(\sigma) := \mathrm{nf}(n\alpha)$ (Lemma 19) and continue as for [Tail-split].

  Why: a sealed program stands for a computation that was stuck on unknowns; at the instance it computes to a definite value, and α records that the fresh σ the checker put in its place stands for that value.

- **[Tail-prop].** With one arm, its field places have the field types, which are true because the scrutinee's type is; extend (Inv). With no arms, the scrutinee's type is `False` and (Inv) would make it true, so no ground instance reaches the node: ex falso. **New beyond F:** a match on a proof of a several-constructor inductive in `Prop` is erased and every arm is a proof; the scrutinee's type is true, so its meaning is inhabited by some constructor, whose fields' types are therefore true, so that arm's claim applies.

  Why: a proof carries no data, so matching on one cannot make the runs diverge; what it yields is the truth of its fields' types. A match on `False` marks a branch that cannot happen at any real instance, since reaching it would make `False` true.

- **[Tail-end] (a leaf).** Lemma 23 gives a successful ground run of the tail term and, by agreement, a successful pop. The symbolic result is live (reading $\bot$ is an error, a borrow made by [Borrow] is live, a closed-off call's borrow row is live, and [Drop] refuses to end a borrow in flight), so the ground result is a borrow by clause (A3) of agreement (Definition 14: the symbolic path has ended at least the borrows the ground path has). For a lemma, the claim below gives $\vDash A\alpha$; [Def] gives $A \equiv G'$; by Lemma 10, $\vDash G'\alpha$; and $G'\alpha = G\beta$ because α respects the path's refinements and records (Lemma 19), so the goal is true at β by Lemma 21.

  Why: a leaf is where the checker accepted a result. Agreement turns "the checker computed this" into "the real run computed this", and, for a lemma, "the checker proved the goal on this branch" into "the goal is true at β".

Which calls the walk needs, and why each is safe. Lemma 23 needs every call that the ground run makes to complete without error at the instance it is at.

Why: these are the only points where the walk depends on code other than d_k's own body; each is discharged by the induction order chosen above, except calls through function values, which need Q1.

- A call of an earlier definition d_j, $j < k$: by the induction hypothesis (i).
- A recursive call of d_k: [Rec] made the symbolic decreasing argument a strict subterm of the entry value as refined; by Lemma 23 the ground argument is the symbolic one under α, and α respects the refinements, so it is a strict subterm of $\beta(\sigma_j)$ and the pair is smaller.
- A call made while evaluating a type: to an earlier definition, in erased mode, by (i) and Lemma 24.
- **New beyond F:** a call through a function value. Its safety comes from the value being good (Q1), and, for a closure formed in d_k that calls d_k, from [Rec] having checked that call against d_k's entry value (principle R4, Section 5).

Claim: proof terms have true types.

Why: inside a lemma the checker accepted proof terms by typing them. This claim says typing never misrepresents truth at a real instance. It is where consistency lives: a closed proof of `False` *would be a proof term whose type is false.*

At agreeing states with (Inv), if $\Omega_s \vdash t \Downarrow \star : A \dashv \Omega'_s$ for a proof term t, then $\vDash A\alpha$. By induction on the derivation: `refl` has type `True`; a constructor of a proposition has true field types by induction and Lemma 10; a proof place has a true type by (Inv); a proof constant by the theorem's induction hypothesis (ii); a `let` or sequence ending in a proof by Lemma 23 in erased mode; a match on a proof as for [Tail-prop]; a lemma call $L_j(\overline{u})$ because its arguments' types make the hypotheses of the ground instance it is at true (Lemma 22), the pair is smaller, so its goal is true there by (ii), and Lemma 22 transports that truth back. **New beyond F:** `J` and `rewrite` (Section 6.5.7) and proofs that are function values.

The lemmas the walk uses are Lemma 19 (generalisation), Lemma 21 (types agree), Lemma 22 (call types), Lemma 24 (modes) and, above all, Lemma 23 (runs agree), which is where the two paths meet. They are stated in Section 7; the next two sections give what they need.

5 The principles the checker enforces

The extended simulation is justified by a small number of principles, each derived from a model of one part of the calculus (dependent fields, case splitting, observation, recursion, erasure, closing off, locality), and each enforced in the checker at a single place, so that a rule cannot be forgotten by one of several code paths. Every current checker rule corresponds to one of them, and in each area probes looked for a rule the principles demand and the checker lacks. Three such gaps were found on the checker of October 2026, each a closed proof of `False`; all are fixed (Section 9). The principles are stated here because the simulation uses them; none is proved.

5.1 Locality

- (L1) *Names are local.* A running body names only places of its own frame. A closure's captures are copied into its own frame when it is called, and a closure captures no borrow. Global definitions are immutable.
- (L2) *Writes stay in the frame.* A borrow carries its content, so a write through a borrow changes the content inside the borrow held in the running frame; the lender's place does not change until the borrow ends.

- (L3) *Calls exchange only what is passed.* Arguments move into the callee's frame (an owned value is moved out of the caller; a borrow argument carries the caller's loan); the callee returns its result; popping its frame drops its bindings, ending its borrow parameters; [Close] fills only the loans of the call's borrow arguments.
- (L4) *Borrowers sit at or above their loans*, in every run whose effects persist: a loan in frame f is borrowed from f or a frame above it. It follows from no borrows in data, no borrows of borrows, closures capturing no borrow, and [Drop] refusing to let a borrow of a local outlive it. So [Access] and [Drop], which start from a place of the top frame, end only borrowers of the top frame. (L4 fails inside the checker's own private copies, which compute a field's type by binding copies of fields in a scratch frame; those copies are discarded, so nothing leaks.)
- (L5) *Ending is the one channel down.* [End] is how a lender below gets its content back; by (L4) the places a run can change below its frame are exactly the places its frame's borrows were lent from.
- (L6) *Private runs leave no trace.* Statements, and the checker's own scaffolding, run on copies that are thrown away.
- (L7) *Sealed programs are closed, and refinement is not a run.* A sealed program's re-run starts from the empty environment and reads no frame. Refinement substitutes across all frames, but that is the typing judgement acting on the whole state, never the effect of a run.

Enforced: the checker ends a borrower found from a place of the top frame in one function (`endTopLoan`), which asserts (L4) outside private copies; it never fires on the test suite.

5.2 Closing off

- (C1) *Reconstruction.* The sealed text rebuilds the call exactly: the same function value (code and captures), the other arguments as their values at the call point, each borrowed content in a fresh cell. Nothing else of the call point reaches the callee, by (L1) and (L3). So a sealed program is a closed text whose run depends only on what it embeds and on the refinements of the world it runs in.
- (C2) *Projections.* Each output (the result, and each borrowed place's final content) is its own sealed program, and its final read copies. A `Unit`-typed output is `()`.
- (C3) *Holes are parameters.* A returned borrow's final content is a parameter of every fill, because at the generic call it is not decided which borrowed place the returned borrow points into. A fill's run never inspects its hole: an inert loan is stuck at a match and passes through reads and writes unchanged. Filling it is substitution, which then re-runs the fills.
- (C4) *Re-run exactly when an input changes, everywhere.* A sealed program is re-run when a value it embeds is substituted (a refinement, or the fill of a loan) or a neutral its run derives is refined, at every position a sealed program can occupy: the environment, stored types, the types of abstract values, the goal.
- (C5) *No re-closing.* On a re-run the head call is not closed off again, so a sealed program makes progress or stays exactly as it is.
- (C6) *One text, one value.* Within a world, a sealed program's text determines its value (by (C1) and determinism), so two derivations of one text may be identified.
- (C7) *Errors surface.* A re-run that fails is a type error where it was triggered; exhaustion is undecided.
- (C8) *Clean inputs.* Every input is loan-free and whole when the text is made.
- (C9) *Blocks are calls.* A stuck match or `J` is closed off as a call of an anonymous function of its captures, so (C1)–(C8) apply with the captures as arguments.

Enforced: one function makes sealed programs and one re-runs them; substitution and re-normalisation share one traversal of every value position of the state (`mapState`), which closed a gap in (C4) where re-normalisation had skipped the types of abstract values.

5.3 Worlds

Checking a match on an unknown σ of an inductive $\mathsf{D}(\overline{a})$ is elimination of a disjoint union: σ denotes an element of $[\![\mathsf{D}(\overline{a})]\!]$, which is the union of its constructors' field tuples, and each arm is one summand.

- (X1) *Worlds are exclusive.* No fact established while checking arm i that depends on σ being C_i is visible while checking arm j: a refinement, a type refined by it, a generalisation record, a captured value.

- (X2) *Refinement is total substitution.* Entering a world substitutes σ everywhere it can occur (environment, stored types, types of abstract values, goal, values embedded in sealed programs and closures) and re-normalises. A component skipped keeps a stale, pre-refinement value in a refined world.
- (X3) *Records are scoped.* A generalisation record $n := \sigma$ is in force in the world that made it and its sub-worlds (private copies, nested splits), nowhere else: the same text n can denote values of different types in different arms, since it may mention the scrutinee.
- (X4) *The block is checked unrefined.* What follows the match is checked once with σ unknown; its sealed programs resolve when a later split learns σ.

What a world may keep is exactly what holds in every world: fresh-name counters (a counter states no fact) and the types of abstract values created inside the arm (nothing kept names them, since the arm's records are dropped with it). An η-refinement of an abstract value of a one-constructor type is *not* among them. Its constructor follows from the type alone, but its fields are fresh abstract values whose types are read off σ's type in the arm's world, and can mention what the arm refined: a value parameter, or an earlier field of a dependent constructor. Kept, a sibling arm reused fields typed in another world, a closed proof of `False` (EtaBoom, Section 9). So an η-refinement, like a generalisation record, lives until the arm that made it ends, and survives the private copies made within it. *Enforced:* every arm of every split runs inside one combinator (`inWorld`), whose exit restores every component of the state except those listed, so a new component is restored by default.

5.4 Wholeness

A dependent constructor $\mathsf{C}(g_1 : A_1, ..., g_k : A_k)$, with A_j mentioning earlier fields, denotes a *Σ-type*: the set of tuples whose later components lie in sets determined by the earlier ones. A borrow of a field, as in Aeneas (the Rust verification tool whose borrow model Ochr's machine follows), receives back when it ends any value of its type.

- (W1) *Open only while the opener runs.* A value may break its telescope only while the code that opened it runs. Wherever it passes to code that relies on its type, it must be whole: read, moved, copied or borrowed whole; captured by a closure, Π-type or block; observed by `Id`; at the end of a borrow of it (a borrow parameter's at its function's return); as a result.
- (W2) *Producer obligation.* The generic path treats every neutral of a dependent type as whole: [Repack] does not look inside it, and [Split] splits it by its telescope. So whatever produces such a neutral must have checked whole the value it stands for. A parameter's σ: the caller passed it whole. A field of a refined neutral: whole because the neutral is. A generalised sealed program: inherited. A stuck call's result or fill: the callee's body checked at its return. A fill with a hole: the same, with the lent part unknown. A stuck block or closed-off `J`: each arm, or the transported term, checked at its end. A hole `?`: an axiom, excluded.
- [Return] is the common form: where a checked body returns, its result and every place it holds by borrow must be whole; a part still lent there is any value of its borrow's type.

Enforced: one function forms every block, taking its arms as computations and running them itself, and calls the same return check that function bodies use; so no block can be formed without the check.

5.5 Erasure and captures

- (E1) *Discard.* Every erased term runs on a private copy of the state it would start in, and the copy is thrown away.
- (E2) *Decided by declaration.* Whether a term is erased is read from syntax and declared types. Since sorts are syntactic, "is a proof" is a property of the term itself, so a re-run of a sealed program erases exactly what the direct arm erases.
- (E3) *Only copy-independent facts survive a copy:* name counters, the types of names created in it, generalisation records (keyed by closed program texts, which mean the same in any state), η-refinements (the fields' types are the copy's world's, which the copy shares), checked recursion obligations, caches.
- (E4) *In place means within the statement's copy.* A borrow-typed argument of a statement runs in place, not on its own copy, but the statement call is itself erased.

- (E5) *Reads move inside statements as at runtime,* on the copy only: a side that reads x and then observes x sees $\bot$, and its equation is stuck.
- (K1) *Captures over-approximate.* For each outer place and each arm, a block's capture mode is at least the arm's effect on it, in the order none $\sqsubseteq$ copy $\sqsubseteq$ write or borrow $\sqsubseteq$ move, measured at the granularity of the place moved and net of the arm's own refills; a closure or Π-type in an arm moves what it mentions.
- (K2) *Over-approximation is safe.* A stricter generic call only removes options for the code after the block: a read of a place the block moved is an error, and an equation over it is stuck.
- (K3) *Effects are measured where they happen:* a move inside a nested block is that block's, and a move inside a statement is discarded with its copy. By (E2) a block's re-run decides as the direct arm does.

Enforced: statement positions run through one combinator (`inStatement`) and the checker's scaffolding through another (`privately`); captures are computed by one function (`blockCaptures`).

5.6 Recursion

A definition **fix** $f\big(\overline{x} : \overline{A}\big) : B$ **by** $x_j \coloneqq b$ denotes the function defined by well-founded recursion on the size of the entry value of x_j.

- (R1) *The measure is the entry value, as refined.* Each recursive call's argument (its content, through a borrow) is compared with the caller's entry value as refined so far, whatever the body did to the place since. Strict subterm implies smaller size. Refinement only adds constructors, so a strict subterm at the generic call is one at every instance.
- (R2) *Recursive calls are named.* f is reached only through the name it binds for itself, or a nested function's capture of that name; which calls recurse is a fact about syntax, never decided by comparing function values, which refinement changes.
- (R3) *Check before running.*
- (R4) *One order.* A definition sees only earlier definitions and itself, so every chain of calls is ordered lexicographically by (definition, entry size). A call through a function value is not a recursive call unless its head is the name; a closure formed in f's body that calls f is checked against f's entry value where it is formed.
- (R5) *Types recurse like terms*, and a statement never mentions the definition being checked.
- (R6) *Runs only unfold.* Untyped runs unfold definitions without the check; they decide nothing about well-foundedness, and a run that does not terminate is exhaustion, never a decision.

Enforced: one function decides whether a call belongs to a recursion entry, by a mark on the binding (`recFrameOf`).

5.7 Observation

- (O1) *Independence.* The two sides of `Id` run from the same state on independent copies, in the same world.
- (O2) *Resolved, not represented.* Observations compare resolved values, so where a borrow sits does not matter; owners are observed, not borrows.
- (O3) *Footprint sufficiency.* A position whose resolved content cannot differ between the sides need not be observed. It can differ only if a side writes it, directly or through a borrow, so the owners of every place written or borrowed, and of every borrow variable handed on whole, are observed; a place only read is not (a read that moves leaves $\bot$, which must be refilled before the borrow over it ends, and refilling is a write).
- (O4) *All owners.* A loan can sit in several owned positions (a hole in several fills); each is observed.
- (O5) *One order.* The conjunction is ordered by syntax, the same on both paths, since $\wedge$ is not commutative by conversion.

Footprints at two paths. A footprint's roots are read off syntax, so they are the same on both paths; only the owners can differ. At the generic call a borrow parameter's loan sits in one fresh cell; at an instance it may sit in several owners (O4). Every extra owner's resolved content is a function of what was written through the borrow plus parts no side writes, so if the generic statement proves the written values equal, each extra owner's equation is true too. That conversion *computes* those equations to `True` is part of stability (Lemma 31).

6 The simulation relation, beyond the fragment

6.1 The fragment F

The paper proves Target 11 for a fragment F defined by six restrictions:

- (F1) *Data:* only `Nat`, `Unit`, `False`, `True`, `And`; data types `Nat` and `Unit`; borrow types `&Nat`, `&Unit`. No pairs, no user inductives.
- (F2) *First order:* top-level definitions with bodies; parameter types data, borrow types or propositions; no `fix` or λ inside bodies, no Π-values, no function parameters, no type families, no `Prop`-valued functions; a definition mentions itself only as a call head in its body.
- (F3) *Propositions:* `True`, `False`, `And`, `Eq` $D\ a\ b$, `Id` $D\ t\ u$ with $D \in \{\mathsf{Nat}, \mathsf{Unit}\}$ and match-free terms.
- (F4) *No stuck blocks:* a match on data only in tail position, so only [Tail-split] and [Tail-gen] split.
- (F5) *Proof forms:* `refl`, pairs, proof places, lemma calls, `let`/sequence ending in a proof, matches on proofs of `And`/`True`, `match h {}` on `False`. No `J`, `rewrite` or `split`.
- (F6) *No move out through a borrow:* a read under `*` is written `clone(…)`.

6.2 Agreement and its assumptions, in F

Definition 14 (agreement, in F). Let Ω_s be a state on the symbolic path, α a valuation that covers and respects it, and Ω_g a ground state with the same frames, bindings and temporaries. $\Omega_s \approx_\alpha \Omega_g$ holds when: (A1) $\Omega_s\alpha$ is defined (every re-normalisation it triggers ends without error); (A2) $\rho(\Omega_s\alpha) = \rho\big(\Omega_g\big)$ up to a renaming of loans; (A3) every position that holds a borrow in Ω_s holds a borrow in Ω_g; (A4) where Ω_s holds borrow_ℓ and Ω_g holds $\mathrm{borrow}_{\ell'}$, $\mathrm{owners}_{\Omega_g}(\ell') \subseteq \mathrm{owners}_{\Omega_s}(\ell)$; (A5) where Ω_s holds $\mathrm{borrow}_\ell w_s$ and Ω_g holds $\mathrm{borrow}_{\ell'} w_g$, $\rho_{\Omega_s\alpha}(w_s\alpha) = \rho_{\Omega_g}\big(w_g\big)$.

(A2) is stated up to resolution because the stronger form is false. With `Pick(n, x, y) := match n { Z ⇒ x, S _ ⇒ y }` and `n` abstract, in `let r = Pick(clone(n), &a, &b); let z = b; …` the hole of `Pick`'s returned borrow sits in the fills of both `a` and `b`, so reading `b` ends `r` on the symbolic path, while at `n = 0` it does not: the states agree only once every borrow is ended. (A3) says the symbolic path has ended at least the borrows the ground path has; (A4) that its owners over-approximate; (A5) fixes what each live borrow holds, without which a borrow could point at different parts of its target on the two paths. The consequence the walk uses: at agreeing states, a place readable on both paths has the same resolved content, so a scrutinee has the same head constructor on both.

The fragment's proof assumes two things, which the plan must discharge for the whole calculus.

> **Quick explainer: naturality.** A *naturality* property says that two ways around a square of operations agree: doing f then g gives the same as doing g' then f'. Here the two operations are running a term and applying a refinement α (substituting values for unknowns). Run on unknowns and then substitute, or substitute first and then run: naturality says both give the same result.
>
> $$(\text{run } \Omega_s)\alpha \quad = \quad \text{run } (\Omega_s\alpha)$$
>
> Ordinary type theory gets this from its substitution lemma. Ochr needs it only *up to resolution*, because the symbolic run leaves open borrows and sealed programs that the run at the instance never creates. The name comes from natural transformations in category theory, but nothing beyond the square is used here.

Assumption 15 (naturality, step by step, in F). Let $\Omega_s \approx_\alpha \Omega_g$, and consider one step of a rule that succeeds from Ω_s on the symbolic path. (N1) *Primitive steps* ([Access], [Read], [Copy], [Move], [Borrow], [Assign], [Clone], [Ctor], binding and dropping, pushing and popping frames): the same step succeeds from Ω_g, and the resulting states, with the step's value as a temporary, agree. (N2) *Calls that close off:* at agreeing call points, if the symbolic call closes off and the ground call runs to completion without error, the resulting states and results agree. (N3) *Resolution commutes with refinement:* $\rho(\Omega_s)\alpha = \rho(\Omega_s\alpha)$ up to a renaming of loans, including through re-normalising a fill whose hole is inert.

Assumption 16 (invariants of the machine). (T1) *Well-formedness:* the rules preserve (WF1)–(WF5). (T2) *Frame:* a run changes the environment below its own frame only through the loans held by that frame's borrows,

and a call's effect is its isolated run, plugged back into the caller. (T3) *Order independence:* in a well-formed state in which no borrow holds a moved-out $\bot$, resolution exists, ending the held borrows one at a time in any order reaches it, and ending some of them first does not change it. (T4) *Injectivity of contexts:* a context that holds a loan is injective in the loan's final value.

Under these, the paper proves Target 11 for F. Its known risks are the preservation of (A3) after a [Drop] (an early symbolic end must be *deep*: no reborrow outlives a borrow the symbolic path ended early), the preservation of (A5) by [Close], and (N3) through re-normalisation with inert loans.

6.3 Agreement for the whole calculus, proposed

The plan extends agreement clause by clause, one feature at a time (Section 6.5). The proposal below is the target; each new clause is tied to an open design question where its exact form is not settled.

Definition 17 (agreement, proposed). Let Ω_s be a symbolic state, α a refinement that covers and respects it and is ground on every abstract value of Ω_s that is not introduced inside the current world, and Ω_g a ground state with the same frames, bindings and temporaries. $\Omega_s \approx_\alpha \Omega_g$ holds when (A1), (A3), (A4), (A5) of Definition 14 hold and: (A2$_\sqsubseteq$) $\rho(\Omega_s\alpha) \sqsubseteq \rho\big(\Omega_g\big)$ up to renaming of loans and of fresh abstract values, where $\sqsubseteq$ is equality except that a sub-value $\bot$ on the left may be any value on the right (Q2); (A6) every function value of Ω_s, under α, is convertible with the function value at the same position of Ω_g, and that value is good (Q1); (A7) every neutral of a dependent type in Ω_s denotes, under α, a whole value, and a value open in Ω_s is open in Ω_g in the same frame (Q5); (A8) every stored type of Ω_s, under α, is convertible with the stored type at the same position of Ω_g; (A9) α respects every refinement and every generalisation record visible in the current world.

6.4 Naturality, generalised

Lemma 18 (naturality, step by step). Let $\Omega_s \approx_\alpha \Omega_g$ (Definition 17). For one step of any rule that succeeds from Ω_s on the symbolic path: (N1) *Primitive steps,* as in Assumption 15, now including closures, constructors of every inductive, field places, and the opening and repacking of dependent values. (N2) *Calls that close off,* including calls with a neutral head (an abstract function). (N3) *Resolution commutes with refinement.* (N4) *Blocks.* If the symbolic path closes a stuck match (or `J`) off as a block, and the ground path runs the arm that α selects (or the transported term), the resulting states agree. (N5) *Erased runs.* A step in erased mode leaves agreement as it found it, on both paths.

How each clause reduces.

- (N1) follows from the decisions commuting with refinement (Lemma 27) at the step, and, where the step does not inspect an unknown, from (P) (Lemma 31).
- (N2) is (C) (Lemma 32) at a call point, with (T2) placing the call's effect in its context.
- (N3) is (P) applied to holes (by (C3) a fill never inspects its hole) together with (T3).
- (N4) is (C) for blocks by (C9), with (K1)–(K2) making the block's captures at least the arm's effects (so the symbolic state holds $\bot$ only at places some arm moves, which (A2$_\sqsubseteq$) allows), (W1)–(W2) making every borrowed capture whole at the arm's end, and (Sp) (Lemma 33) making the arm the ground path takes the one checked in the world of its constructor.
- (N5) is (E1)–(E5): each path discards the copy, and what survives a copy holds in every state.

6.5 Feature by feature

6.5.1 User inductives, pairs and dependent constructors (lifting F1)

What changes. Ground values are trees of constructors; `eq` uses injectivity and disjointness; field places reach into any constructor. **Clauses.** (A8) is needed because observed places now have types that refinement can change (a field type read at the scrutinee's parameters); the footprint lemma's clause that observed places have the same type on both paths becomes "convertible types under α". **Principles.** (V2), (I1)–(I9).

> **Open design question Q4 (early ends are deep, with pairs).** In F, an early symbolic end of a borrow comes from a hole inside a fill: [Access] or [Drop] meets the loan inside a sealed program, the ended borrow's

content (reborrows included) is substituted into that same fill, which stays stuck by (C3), so the same access keeps ending loans until none remains and ends the reborrows too. This argument preserves (A3). The paper notes that pairs break it: a shallow read of one field could leave a reborrow in the other field alive on the symbolic path while the ground path ends it later. The current rules include pairs and every user inductive, so the plan needs a new argument for (A3), or a weaker (A3) that the later steps tolerate. The rule that an assignment's [Access] ends only the loans in the part of the place's content that the place owns, and the rule that a match's [Access] ends every loan inside a neutral head, are the likely ingredients.

6.5.2 Closures, λ and function parameters (lifting F2)

What changes. Parameters can be functions (abstract functions σ of a Π-type, refined to closures); bodies form closures; conversion compares functions by their generic observations ([Conv-fun]); a call through a neutral head closes off at once. **Clauses.** (A6). A closure's captures are values with no borrow (WF4), so agreement of captures under α is ordinary value agreement. **Principles.** (L1), (C1) with the function value embedded, (V5), (R2), (R4). **What it needs.** The calls through function values in Section 4.4 are safe only if the function value is good, which is Q1. The modes lemma (Lemma 24) in F used that runtime and erased runs differ only in the rule for a call through a place (a call does not consume its function), which concerns function values; beyond F that difference must be shown harmless.

6.5.3 Matches inside statements and stuck blocks (lifting F3, F4)

What changes. A stuck match not in tail position is closed off as a block on the symbolic path, while the ground path runs the arm the value selects, in place. Generalisation records can now come from [Split-gen] inside statements and from **split** f **in** t, not only from [Tail-gen]. **Clauses.** (A2$_{\sqsubseteq}$): by (K1) a block takes each place in the strongest mode any arm needs, so the symbolic state can hold $\bot$ where the ground arm left a value. By (K2) no later code reads such a place, and an equation over it is stuck. (A9) with (X1)–(X4): a record made in an arm is dropped when it ends, so α need respect only the records of the current world. **Principles.** (C9), (K1)–(K3), (X1)–(X4), (W1)–(W2), [Return].

Open design question Q2 (agreement after a block). (A2$_{\sqsubseteq}$) weakens (A2) to an ordering in which a symbolic $\bot$ matches any ground value. Every use of (A2) in the fragment's proof must be rechecked against the ordering: the footprint lemma (a ground footprint position whose symbolic owner was moved by the block must be one the statement only reassigns, or one whose equation is stuck), the types-agree lemma for `Id` (a stuck equation over $\bot$ on the symbolic side, a computed equation on the ground side), and the consequence that readable places agree (a place $\bot$ on the symbolic side is not readable there, so the walk never meets it). Whether the ordering must also cover a borrow that the block took by `&` and that the ground arm only read is part of the question.

6.5.4 Returned borrows and holes, with blocks

What changes. F already has returned borrows. Beyond F a block's arms can return borrows, a returned borrow can be moved into a second call or reborrowed into a stuck call, and a hole can sit in the fills of a block's captures. **Clauses.** (A4) and (A5) as in F; their preservation by [Close] is (C) together with (C3). **Principles.** (C3), (O4), and the injectivity relation R (Definition 7), which (T4) generalises: a function value's backward function must be injective, and a context holding a loan must be injective in the loan's final value. For function values formed by accepted code, injectivity is a property their body must be shown to have; for abstract functions it is part of being good (Q1).

6.5.5 Dependent fields

What changes. Values can be open; [Repack] checks wholeness at the points of (W1); neutrals of dependent types are trusted to be whole (W2). **Clauses.** (A7).

Open design question Q5 (dependent values in the relation). (A7) asks that every neutral of a dependent type denote a whole value under α, which is (W2) read semantically, and that openness match across the paths. But the paths can differ in how an open field is typed: by its content on one path and by its telescope on the other, when the two types are convertible on one path and not the other. The clause must say what an open value means in the model (a tuple whose fields each lie in *some* type, not necessarily the telescope's) and how the two paths' typings of it relate. A proposed simplification, typing an open data field always by its content, would remove one value-dependent decision and may make this clause simpler; it is a rule change and has not been adopted.

6.5.6 Fording and value parameters

What changes. Inductives can take value parameters, have equation fields, and recur at other arguments. A case that cannot arise receives an equation that computes to `False` once its index is split. **Clauses.** None new. Arguments at value parameters are erased positions (E1); equation fields are proof fields ($\star$), so they never block `eq`; an impossible arm's `match h {}` is the [Tail-prop] zero-arm case, vacuous under (Inv). **Model.** Definition 6′s simultaneous least fixed point over the parameter family, and (I2).

6.5.7 `J` and `rewrite` (lifting F5)

What changes. Transport $\mathsf{J}(A, a, b, P, h, t)$ computes to t only when $a \equiv b$; otherwise it is closed off as a block whose one arm is t. **rewrite** h **in** t is `J` with the motive read off the goal. **Clauses.** A `J` block is a block (N4). **Principles.** (V8), [Return] for the transported term.

Open design question Q3 (stuck equations and `J` at ground instances). The fragment counts a stuck equation as true, which is safe there because a stuck equation arises at a ground state only from $\bot$, and no proof form eliminates it. Beyond F this is false: with congruence (`cong`), a hypothesis $\mathsf{Eq}(\Pi(n : \mathsf{Nat}).\ \mathsf{Nat}, f, g)$ between two extensionally different closures is stuck, and if it counted as true, `cong` would derive $\mathsf{Eq}(\mathsf{Nat}, f(0), g(0))$, which computes to `False` at a ground instance. So truth must be semantic (Definition 8), with only the $\bot$ case of a stuck equation treated specially, or not at all. A second consequence: at a ground instance a hypothesis $\mathsf{Eq}(A, a, b)$ can be true while $a \not\equiv b$ (two closures with equal meanings and different code), and then a `J` in a data function's body is stuck on the ground path. Then Target 11 (i) "runs to completion" fails. Either ground runs must be shown never to reach `J` at true-but-unconvertible endpoints (plausible for closed runs, whose hypotheses are closed proofs, which can only be `refl` at such a type), or (i) must be weakened to "does not go wrong".

6.5.8 Moves through borrows (lifting F6)

What changes. A read under * may move out (`let v = *x; *x := S(v)`), so a borrow's content can hold $\bot$ until it is refilled; a borrow must be whole again when it ends.

Open design question Q6 (resolution with moved-out contents). The fragment's resolution lemma ("no borrow ever holds a moved-out $\bot$") is what makes resolution defined and (T3) applicable at every state the walk visits. Beyond F6 it becomes "every borrow is whole when it ends". Resolution is then not defined at a state where some borrow's content holds $\bot$, and the walk visits such states. Either resolution must be extended ($\bot$ resolves to $\bot$, and ($\mathrm{A2}_{\sqsubseteq}$) absorbs it), or agreement must be stated only at states where every live borrow is whole, with the walk's steps grouped so that it never stops between a move out and its refill.

6.5.9 Universes, type families and large elimination

What changes. Types are values that runs compute, including types computed by recursion on data (`Cells(E, n)`); large elimination from subsingletons; `Prop : Type₁`. **Clauses.** (A8). The generalisation lemma (Lemma 19) now covers records made at any type, so the type of a generalised value is its sealed program's type in the current world, which (X3) keeps from leaking. **Model.** Inaccessibles per level; a type family computed by recursion denotes the corresponding function into a universe.

6.5.10 Erased positions

What changes. Every statement position (types, proofs, erased calls and their arguments, `J`'s endpoints and motive, the sides of `Eq`, the observations of `Id`) runs on its own copy. **Lemma.** (N5) and the modes lemma (Lemma 24): a data function's erased run from a ground instance completes with the same result as its runtime run. Beyond F the only difference between the modes is the rule for calls through a place (a call does not consume its function, and a closure may not move its captures), which must be shown not to change a completing run.

6.5.11 Recursion

What changes. Recursion through closures, through nested functions and block arms, and on value parameters' data. **Principles.** (R1)–(R6). The walk's measure is (R4)'s. The induction needs, beyond F, that the symbolic case tree covers the path the ground run takes, which is (Sp) (Lemma 33), and that a recursive call inside a closure called later is still checked against its own entry (R2), which is how the third closed proof of `False` (Section 9) arose before it was fixed.

7 The lemmas and how they depend on each other

This section states every lemma the plan uses, with its status and what it depends on. Figure 1 draws the dependencies and Figure 2 lists the statuses.

7.1 The walk's lemmas

Lemma 19 (generalisation). Let a state Ω_s be reached on the symbolic path, and let α cover and respect every abstract value of Ω_s except those introduced by the generalisation records visible in the current world. Extend α by $\alpha(\sigma) := \mathrm{nf}(n\alpha)$ for each visible record $n := \sigma$, in the order they were made. Then (a) the visible records are exactly those made on the path inside the current world; (b) $\alpha(\sigma)$ is a value of type $\Delta(\sigma)\alpha$; (c) $(\Omega_s[\sigma/n]^+)\alpha = \Omega_s\alpha$, so generalising is invisible under α.

Argument. (a) is (X3) with (E3): a record survives a private copy but not its arm. (b): n occurs in the state, so its normal form under α exists by (A1); it is a value of the type of the place it was generalised at, which is its sealed program's type in the current world. (c): every occurrence of n became σ and every re-derivation normalises to σ (C6); under α both become $\mathrm{nf}(n\alpha)$, which is what n becomes, by determinism. *Status.* **paper argument** in F (records only from [Tail-gen]); beyond F **reduced to (X1)–(X4), (C6)**.

Lemma 20 (footprints). Let $\Omega_s \approx_\alpha \Omega_g$ and let t, u be statement terms whose typed runs from Ω_s succeed, with footprints W_s and W_g at the two states. Then (a) every position in W_g but not W_s is owned only through borrow variables that hold $\bot$ at Ω_s, which t and u only assign as whole variables; (b) for each position in exactly one footprint, the ground runs of t and u leave its resolved content unchanged, or one of them moves it out, so its equation is stuck; (c) each observed place has convertible types on the two paths under α.

Argument. By (T2) and (O3), a run changes a position only by writing it or writing through a borrow it owns, which puts it in the footprint. (a) by (A4) and the error rules for a variable holding $\bot$; (b) by the frame fact; (c) by (A8). *Status.* **paper argument** in F; beyond F **reduced to (T2), (A8), Q2**.

Lemma 21 (types agree). Let $\Omega_s \approx_\alpha \Omega_g$ and let A be a proposition. If every call made in erased mode by the ground evaluation of A completes, and A evaluates to T_s at Ω_s, then it evaluates to some T_g at Ω_g and $\vDash T_s\alpha$ iff $\vDash T_g$.

Argument. By induction on A: connectives by induction; `Eq` because the sides' runs agree (Lemma 23) and loan-free values in flight are the symbolic ones under α; `Id` because each observation under α equals the ground one after resolution, position by position, and the conjuncts for positions in only one footprint are true or stuck (Lemma 20); Π-types and user inductives through Definition 6. *Status.* **paper argument** in F; beyond F **reduced to Lemma 23, Lemma 20, Lemma 10, Q3**.

Lemma 22 (call types). Let L be a lemma called at a ground call point with arguments $\overline{w}$, and β the ground instance the call is at. Each parameter type of L, and its codomain, are true at the call site iff they are true at β.

Argument. Apply (T2) with the parameter frame as the core: each run made while evaluating a type at the call site is the run at the instance, with the final contents of the borrowed cells plugged into the caller's places. An `Id` conjunct over a borrow parameter's cell at the instance compares the final contents of the cell; at the call site, of the owners of the loan, which at a ground state is one owner, holding the borrow's final value in a context K; by (T4) K is injective, so the equations have the same truth. *Status.* **reduced to (T2), (T4)**.

Lemma 23 (runs agree). Let $\Omega_s \approx_\alpha \Omega_g$, and let t be a term, not a match in tail position, with $\Omega_s \vdash t \Downarrow v_s : A \dashv \Omega'_s$ in runtime or erased mode. Suppose every call the ground run of t makes completes without error at the instance it is at. Then the ground run of t terminates without error, the states agree at every call point (each ground argument being the symbolic one under α, its content for a borrow), and the final states, with the values as temporaries, agree.

Argument. By induction on t, one step at a time, using Lemma 18 for each step. At a call, the arguments agree; if the symbolic call closes off, (N2); if it returns, its body's steps are (N1) steps in the callee's frame. At a stuck match or `J` not in tail position, (N4). Termination: the ground run visits each subterm finitely often (no loops; recursion only through calls, which complete by hypothesis), and performs finitely many [End]s at each access. *Status.* **paper argument** in F, conditional on (N1)–(N3); beyond F **reduced to Lemma 18, Q1–Q6**.

Lemma 24 (modes). If a data function's runtime-mode run from a ground instance completes without error, so does its erased-mode run, with the same result.

Argument. Reads in statements move as at runtime (E5), so the runs take the same steps, except for the rule for calls through a place, which concerns function values. *Status.* **paper argument** in F; beyond F **open**.

Lemma 25 (well-founded recursion). Every chain of calls made by the ground runs of an accepted program from good ground instances is finite.

Argument. (R4) orders every call by (definition, entry size); (R1) and (R2) make each recursive call decrease it, provided the symbolic case tree covers the path the ground run takes, which is (Sp). Calls through function values are finite because good values terminate (Q1). *Status.* **reduced to (R1)–(R6), Lemma 33, Q1**.

7.2 Stability: the two paths agree

Lemma 26 (stability). For every reachable state Ω, refinement α respecting Ω, and term t: (decisions) the checker's decision functions, given the refined state and term, decide what they decided on the originals (Lemma 27); (runs) if t runs at Ω to (v, Ω') on the symbolic path, the run of $t\alpha$ at $\Omega\alpha$ does not fail, and if it finishes with (v'', Ω''), then $(v\alpha, \Omega'\alpha)$ and (v'', Ω'') agree once every borrow is ended and the abstract values either run created are renamed injectively onto the other's.

In Lean this is `Ochr.Stability.Conjecture2 := DecisionsCommute ∧ RunsCorrespond`; its proof, `stability`, has the decision fields of Lemma 27 that are proved filled in and `sorry` elsewhere. The (runs) half is the whole-run form of what the walk uses step by step (Lemma 18); both reduce to the same lemmas, N = (P) + (C) + (Sp), and (M).

Lemma 27 (decisions commute). For every decision function d of the checker, state Ω and refinement α respecting Ω, on α-refined inputs at $\Omega\alpha$ d decides what it decided on the original inputs at Ω. The decisions, with their status: (d1) *erasure flags* of a term: **reduced to Lemma 28, and two invariants of the kind of (d0)** (d2) *whether a match is on a proof:* **proved in Lean** as `decisionsCommute_byType`; it reads the arms' constructor names, never their bodies (`byTypeMatch_arms`); (d3) *a function type's class,* for Π-type values, generic to instance: **proved in Lean** where the codomain's shape fixes the class (`decisionsCommute_fnClass_shape`), assuming the class cache is sound (the checker memoises each Π-type's class, and soundness means a memo always agrees with recomputing it); in general **reduced to Lemma 28** (d4) *[Close]'s row,* for function values (not blocks): **proved in Lean** as `decisionsCommute_closeRow`; (d5) *a block's captures and modes:* **reduced to (runs) half** (d6) *the type an observed place is compared at,* up to conversion: **reduced to (runs) half, Lemma 34** (d7) *footprint containment,* $W_{\Omega\alpha}(t\alpha, u\alpha) \subseteq W_\Omega(t, u)$: **reduced to Lemma 29**, proved as `decisionsCommute_footprint` given it. The fact everything signature-reading rests on is also proved: (d0) *refinement keeps the signature* (the declared

inductives and globals), so every decision reading only the signature is stable: **proved in Lean** as `onState_sig`, with `sigDecisions_commute` (whether a constructor or field is a proof, large elimination, matches by type) and `readsSig_refineDecl`, `readsSig_preFlags` (a binding's class and the erasure flags read from a declared reading).

Two of the fields hold only in the generic-to-instance direction and only for Π-type values: for an unknown A : `Type` that a refinement makes a Π-type, the generic run reads no class while the instance reads one. No accepted program reaches this, because every call checks that its callee's type is a Π-type before reading its class. The [Close] row of a stuck block is excluded from (d4): refining an unknown match type to `Unit` turns the row from data to `Unit`, which is harmless (a value at `Unit` is `()` on both paths) and belongs to (C).

Lemma 28 (declared readings survive refinement). If an embedded value v has declared reading d at Ω, then $v\alpha$ has declared reading d at $\Omega\alpha$.

This is the statement that evaluation preserves declared types (sorts are syntactic), applied to the values refinement re-runs. It is **false at unchecked globals**: with `F (n : Nat) : Nat := Nat` (not accepted, but constructible as a state), the sealed program $\lceil$`F(σ)`$\rceil$ reads as data, and after $\sigma :=$ `Z` it re-runs to `Nat`, which reads as a sort. So it is an invariant of states reachable by checked programs, proved by induction over the evaluator together with typing. Lean statement: `ValueDeclCommutes`, with the counterexample as the theorems `probe_d55_*`. *Status.* **open** needs Q7.

Lemma 29 (refinement shrinks the borrow profile). For a refinement α respecting Ω, the top frame of $\Omega\alpha$ has the same bindings as Ω's, each binding of $\Omega\alpha$ is a borrow (by its type or its value) only if it is one in Ω, and its owners are among Ω's.

Given it, $W_{\Omega\alpha} \subseteq W_\Omega$, because substitution keeps every term's free places (`substT_occs`) and footprints are monotone in the borrow profile (`footprint_mono`), both **proved in Lean**. The shape part is provable as (d0) was. The borrow part needs two run invariants: a sealed program of a type never re-normalises to a borrow type, and data never re-normalises to a borrow or gains a loan. These are "`&` is never produced by computation" and (WF1) as invariants of runs. *Status.* **reduced to (T1), (T5)**. Refinements must be borrow-free for it to hold: a refinement giving an unknown type a borrow type would make a plain read count and grow the footprint (theorems `probe_footprint_*`), which is why Definition 1 requires borrow-freeness.

Lemma 30 (N: symbolic naturality). N holds when (P), (C) and (Sp) do:

- (P) *Parametricity in uninspected unknowns* (Lemma 31).
- (C) *Closing off is correct* (Lemma 32).
- (Sp) *Splits commute* (Lemma 33).

Lemma 31 ((P) parametricity). A run that completes without inspecting the abstract value σ (it never matches on it, compares it, or reads through it) computes, with $\alpha(\sigma)$ in σ's place, the same steps on the substituted values, and leaves the substituted state.

> **Quick explainer: reified computations and thunks.** A *reified computation* is a program written as data. Instead of a function that reads and writes the heap directly, it is a tree whose nodes are requests, such as "what does handle h hold?" or "allocate a new handle", each followed by the rest of the computation as a function of the answer. An *interpreter* walks the tree and answers the requests against a heap; this is the "free monad" pattern of functional programming. The point for a proof is that every access to the heap is a visible node: "this run never asked about σ" is a property of the tree, and the fact that such a run cannot depend on σ is proved once, by induction on trees, for every program at the same time. A *thunk* is a suspended computation, a stored "run this call later, on these inputs". Closing off a stuck call becomes allocating a thunk, and re-normalising it after a refinement becomes *forcing* it, that is, running it now.

On the heap slice: **proved on the heap slice**. The slice's evaluator is a *reified computation* (a tree of requests, which an interpreter runs against a heap) over two effects, `view` (ask what a handle holds; an unknown answers nothing) and `alloc`. `exec_agree`: two heaps with the same allocation counter that differ only on a set D of existing handles give the same result, and final heaps differing only on D, for any computation that never asks about D; it is one induction on the computation tree, proved once. `P_evalH` is `exec_agree` with $D = \{\sigma\}$, with no case of the evaluator inspected. *On the checker:* **open**. Measured on 193 runs, the checker's run at a ground value agreed

with its generic result refined by that value in every case where the heap run never asked about the unknown. A direct proof against the current machine would need an invariant for every one of about 30 state components and a lemma per function of the 116-function evaluator cycle; the estimate recorded with that measurement is 10,000 to 20,000 lines of Lean, and it is not recommended.

Lemma 32 ((C) closing off is correct). Let a call close off at Ω, leaving its fills in the lent places and its result r. For a refinement α under which the direct call at $\Omega\alpha$ runs to completion with final state Ω'' and value v: once the caller's [End] has supplied the hole and every borrow is ended, the refined, re-normalised closed-off state agrees with Ω'', and $r\alpha$ with v, up to renaming of loans.

Reduction. By (C1) the sealed text rebuilds the call, so re-running it under α is running the call in isolation at the refined arguments; by (T2) that is the call in context; by (C3) and (P) supplying the hole afterwards equals running with it; and (C4)–(C8) say the checker re-runs every occurrence, never re-closes, identifies one text with one value, surfaces errors and starts from clean inputs. **reduced to (T2), Lemma 31 on holes**. *On the heap slice*, closing off is a *thunk* (a heap node standing for a delayed call) over the call's input handles, and a re-normalisation is forcing it: `force_is_direct` (forcing a thunk takes exactly the direct call's outcome, given the same allocation counter and agreement on what the inputs reach) is **proved on the heap slice** on branch `heap-close` (file `Ochr/Heap/Close.lean`), assuming that the outcome is invariant under renaming fresh handles and that no input is itself a thunk. On the suite, 579 of 579 forced thunks agree with the direct call.

Lemma 33 ((Sp) splits commute). Where the generic run splits σ, the constructor of $\alpha(\sigma)$ picks the arm the instance takes, and that arm's state, refined by the fields' values, is the instance's.

Reduction. The arm is checked in the world where σ is that constructor (X1), with σ substituted everywhere (X2) and only that world's records in force (X3); refining further by the fields' values is a further substitution. *Status.* **paper argument**, from (X1)–(X4); **open** in Lean.

Lemma 34 ((M) typed and untyped runs agree). A typed run and an untyped run of the same term from the same state agree when neither is stuck.

This is the design claim that typing is the machine run with types; it is used wherever typing re-observes a stuck side or re-runs a callee. *Status.* **open**.

Lemma 35 (the frame property (T2)). A run changes the state below its own frame only through the loans held by its frame's borrows, and a call's effect is its isolated run, plugged back into the caller.

On the heap slice: **proved on the heap slice**. A *capability executor* runs a computation that may view or write only handles it holds, and store only nodes whose children it holds; viewing grants a node's children and allocating grants the fresh handle. Proved: `cap_queries` (a successful capability run asks only about handles reachable from what it started with, or fresh ones), `untouched` (a handle never asked about keeps its node), `frame` (a successful capability run's result is its isolated run's, and the rest of the heap is left as it was) and `call_effect` (a callee holds exactly its arguments' handles, and through a borrow argument reaches the caller's cell and nothing else). On the suite the capability executor refused none of 1,166 runs. *On the checker:* **reduced to (L1)–(L7)**, with (L4) asserted at run time.

Lemma 36 (the other machine invariants). (T1) the rules preserve (WF1)–(WF6); (T3) resolution exists and is independent of the order of [End]s; (T4) a context holding a loan is injective in the loan's final value, and every accepted function value's backward function is injective; (T5) no step produces `&` by computation, and no data value ever holds a borrow or a live loan.

Status. **open** for the current rules. On the heap design a borrow is a reference to a place: there is no resolution, and ending a borrow only clears the place's *lent* mark (a heap checker still needs one per place, since otherwise a write through a borrow that today's checker has ended would go undetected), so (T3) becomes much simpler there, though not trivial.

7.3 The dependency graph

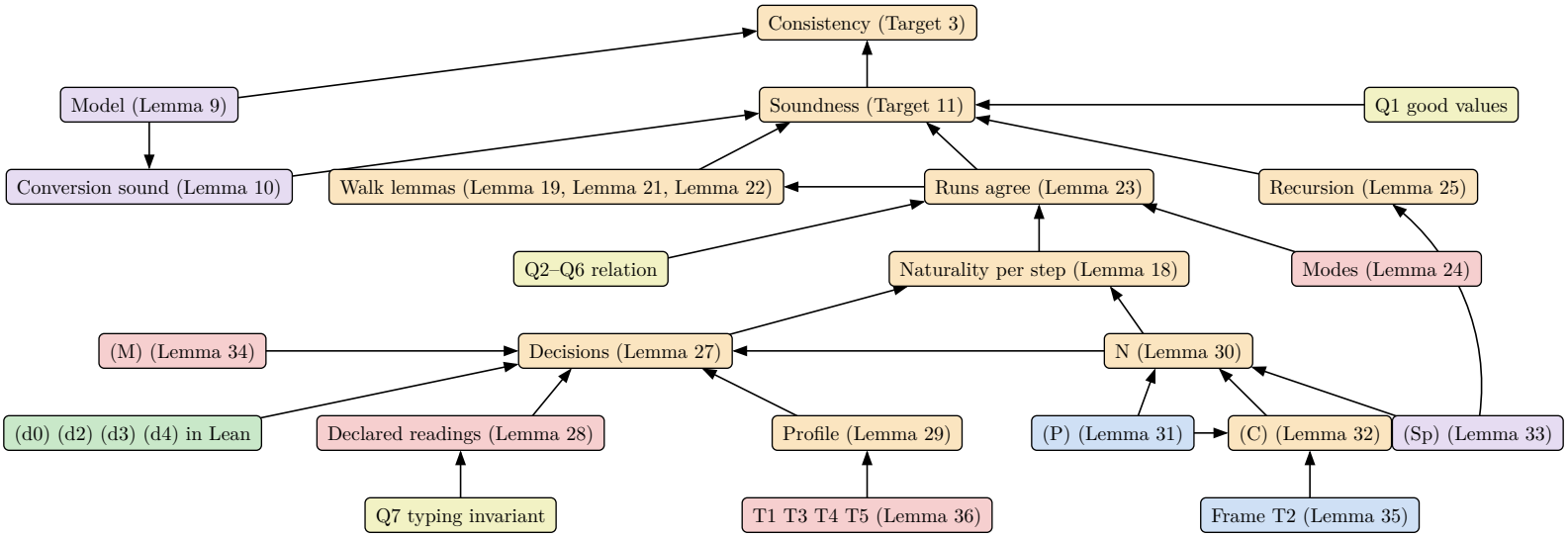


Figure 1: The lemmas and their dependencies; an arrow points from a lemma to one that uses it. Not drawn, to keep the figure readable: conversion soundness also uses (P), and the walk lemmas also use the frame property and (T4). Colours: proved in Lean, proved on the heap slice, reduced, paper argument, open, open design question.

Lemma	Status	Depends on; Lean names
Consistency (Target 3)	**reduced to soundness, model**	Lean statement `Ochr.Consistency.consistency` (`sorry`)
Soundness (Target 11)	**reduced to the walk**	Lemma 23, Lemma 25, Lemma 10, Lemma 19, Lemma 21, Lemma 22; Q1
Model (Lemma 9)	**paper argument**	conditions (I1)–(I9), all enforced by the checker
Conversion sound (Lemma 10)	**paper argument**	(V1)–(V9); (P) for (V4), (V5), (V9)
Generalisation (Lemma 19)	**reduced to (X1)–(X4)**	(C6), (E3)
Footprints (Lemma 20)	**reduced to (T2)**	(A8), Q2
Types agree (Lemma 21)	**reduced to runs agree**	Lemma 20, Lemma 10, Q3
Call types (Lemma 22)	**reduced to (T2), (T4)**	
Runs agree (Lemma 23)	**reduced to naturality**	Lemma 18; Q1–Q6
Modes (Lemma 24)	**open** beyond F	(E5); calls through places
Recursion (Lemma 25)	**reduced to (R1)–(R6), (Sp)**	Q1
Naturality per step (Lemma 18)	**reduced to decisions, N**	(T2), (T3), (E1)–(E5), (K1)–(K3), (W1)–(W2)
Stability (Lemma 26)	**reduced to decisions, N, M**	`Ochr.Stability.Conjecture2`, proof `stability` (partly `sorry`)
Decisions (Lemma 27)	partly **proved in Lean**	`onState_sig`, `decisionsCommute_byType`, `decisionsCommute_closeRow`, `decisionsCommute_fnClass_shape`, `sigDecisions_commute`, `decisionsCommute_footprint`
Declared readings survive (Lemma 28)	**open**	`ValueDeclCommutes`; false at unchecked globals (`probe_d55_*`); Q7
Borrow profile (Lemma 29)	**reduced to (T1), (T5)**	`substT_occs`, `footprint_mono` proved
(P) (Lemma 31)	**proved on the heap slice**	`Ochr.Heap.exec_agree`, `Ochr.Heap.P_evalH`
	open on checker	
(C) (Lemma 32)	**reduced to (T2), (P)**	`force_is_direct` (heap slice, branch `heap-close`), assuming renaming invariance
(Sp) (Lemma 33)	**paper argument**	(X1)–(X4)
(M) (Lemma 34)	**open**	
Frame (T2) (Lemma 35)	**proved on the heap slice**	`Ochr.Heap.frame`, `call_effect`, `cap_queries`, `untouched`
	reduced to (L1)–(L7) on checker	
(T1), (T3), (T4), (T5) (Lemma 36)	**open**	(T3) trivial on the heap design

Figure 2: Status of every lemma.

8 The heap port, and the order of work

8.1 Why a port is on the critical path

(P) is the heart of N, and N is what every model of every area rests on. Against the checker as it stands, (P) is a statement about a 116-function evaluator cycle over a state with about 30 components, and the measured cost of proving it directly is prohibitive (Lemma 31). The *heap design* changes that:

- values are handles into a heap of nodes, and an unknown is a node with no content yet;
- the machine sees a node only by asking (`view`) and creates one only by `alloc`, and is written as a reified computation, so running it is interpreting a tree of `view` and `alloc` requests against a heap;
- a refinement fills an unknown's node in place, instead of substituting through the environment, every stored type and every sealed program;
- closing off allocates a thunk over the call's input handles, and re-normalisation is forcing it;

- a borrow is a reference to a place, so there is no resolution and no substitution of a loan's content; ending a borrow only clears the place's lent mark, which a heap checker still needs (the loan remains, minus the substitution).

Then (P) is one induction on the computation tree, proved once (`exec_agree`); fresh names coincide on both paths, so no renaming is needed for runs (P) covers; locality and the frame property are one induction each for the capability executor (Lemma 35); and (C) reduces to the frame property, (P) on holes, and renaming invariance of fresh handles (Lemma 32).

What blocks the port. (P) covers only runs that never ask about the unknown. The current machine walks whole values in several places (checking for holes, searching for loans, computing owners, testing for borrow types), and a walk asks about every node it passes. Measured on the heap slice over 94 runs with one unknown: (P) applies to 22 runs if reads look only where they go, and to 3 if every read walks the value it reads. So the walks must go first. A first step (branch `port-step1`) replaced the 17 walks that scan the whole environment for a borrow or a live loan with an index from loan labels to holders, kept current on every write, verified against the old scans on the whole suite with no verdict changed; the other 33 walk inside a single value and need a summary on every node (a hole flag, a loan index), which is the port's value layer itself. Two further prerequisites: tables keyed by a value's contents (generalisation records, the conversion stack, about a dozen equality tests) must be keyed by node identity, and every unknown must carry a type that refinement never changes. The heap design also replaces moving borrows and loans with references to places; that the two agree on accepted programs is measured (1,166 runs over 98 functions, no difference), not proved.

8.2 What does not depend on the port

- The set model and its conditions (Lemma 9), and soundness of conversion (Lemma 10), on paper.
- All of the design questions Q1–Q9, and the proposed agreement relation (Definition 17).
- The extended walk (Section 4.4) and the reductions of Lemma 23, Lemma 18, Lemma 19, Lemma 21, Lemma 22, Lemma 25, written conditionally on N, (M) and (T1)–(T5): this is the fragment's theorem redone for the whole calculus, conditional.
- In Lean against the checker: the port-independent parts of Lemma 27 (the scope invariants (d1) needs, the shape part of Lemma 29).

8.3 What does

(P), (C) and the frame property for the checker itself, and probably (M), which may become free if typing is the same reified computation with types as a second output. (T3) becomes trivial on the heap design.

8.4 Recommended order

Estimates below are those recorded with the measurement of each piece, in weeks of focused agent work; no estimate exists for the paper steps.

1. **Settle Q1–Q9** (Section 10). Each decides the form of a definition everything later uses. Q1 (good values) and Q7 (the typing invariant) first, since they fix what the soundness theorem quantifies over and over which states stability is claimed.
2. **Write the model and conversion soundness on paper** (Lemma 9, Lemma 10). They fix what "true" means, which Q1 and Q3 need.
3. **Write the conditional theorem on paper**: Target 11 for the whole calculus, conditional on Lemma 18, (M) and (T1)–(T5), with the agreement relation of Definition 17. This is the first milestone a reviewer can read: the fragment's argument, extended to every feature, with every remaining assumption named.
4. **Finish the port-independent Lean parts** of Lemma 27.
5. **Port the value layer to the heap design**, slice by slice, each step compared with the current checker on the suite: walks to node metadata, identity keys, typed unknowns, then the value layer (estimate: 4 to 8 weeks to parity on every verdict).
6. **On the heap design, in Lean:** (P) and the frame property come with the port; (C) needs renaming invariance of fresh handles (2 to 3 weeks; the port needs it anyway) and inputs that are themselves thunks

(about 1 week); (Sp) as fill and unfill of the scrutinee's node (about 1 week); (M) by a simulation (2 to 4 weeks) or by construction.

7. **The machine invariants** (T1), (T4), (T5), and Lemma 28 under the typing invariant of Q7.
8. **Assemble:** discharge the conditions of the paper theorem, and state the Lean form.

9 Where the plan could fail

The risks below are places where a lemma might be *false*, not merely unproved.

Statements that turned out false as first written. Mechanising the decision half of stability corrected four statements, none of them a checker bug:

- the class and [Close] row of a function type commute with refinement only for Π-type values and only from the generic to the instance (an unknown $A : \texttt{Type}$ refined to a Π-type reads no class generically);
- the [Close] row of a stuck block does not commute (an unknown match type refined to `Unit` flips it from data to `Unit`, harmlessly);
- footprint containment fails for a refinement that makes an unknown type a borrow type (excluded by requiring borrow-free refinements, which is what refinements are);
- declared readings survive refinement only at states of checked programs (Lemma 28).

Each says the same thing: stability holds for states that checked programs reach, not for arbitrary states. The plan must state it relative to a typing invariant (Q7). If that invariant cannot be stated so that refinement preserves it, stability as formulated is false and the decomposition must be redone.

Equality before resolution is false. The `Pick` example (Section 6.2) shows the two paths agree only after every borrow is ended, so agreement must be stated up to resolution, and every step that inspects a borrow's position must be shown not to depend on it.

Early ends with pairs (Q4). The fragment's argument for preserving (A3) explicitly fails for pairs.

Stuck equations and `J` (Q3). Counting stuck equations as true is false beyond F; and a ground run may get stuck at `J`, which would falsify soundness clause (i) as stated.

Impredicative quantification over functions (Q1). If good values are defined by a logical relation, impredicative `Prop` breaks a definition by induction on types.

Closed proofs of `False` found while the principles were being derived. On the checker of October 2026, three accepted programs proved `False`, and a referee of this plan found a fourth on 2 November; all are fixed and are now rejected tests:

- writing through a returned borrow of an index field, so that a value's length and its dependent field disagreed while the generic path trusted the value as whole (fixed: a part still lent at a whole-again point is an unknown of its type);
- a stuck `J` closed off without checking its borrowed places at the end of its transported term (fixed: a `J` block is checked at its return, like any block);
- a recursive local function capturing a value it splits on: the split rewrote the capture, the recursion check identified calls by comparing function values, and missed them (fixed: recursive calls are recognised by name);
- `EtaBoom` (Section 11): leaving a split arm kept the η-refinement of a one-constructor unknown, whose fields' types had been read in the arm's world, so a sibling arm reused a field typed `Void` where a `Nat` was due (fixed: an η-refinement, like a generalisation record, lives until the arm that made it ends).

The first three were missing producer or recognition obligations; the fourth was a fact kept beyond the world in which it held. The principles of Section 5 now name the obligations and single points in the checker enforce them, and probes of five further areas found nothing; but a list of principles can be incomplete, which is what the plan exists to rule out. Independent evidence: a route that translates each accepted program into Lean and has Lean's kernel check it refused, for the second and third programs, the certificates that a checker still accepting them produced, each for the actual reason the program was wrong.

Incompletenesses are not risks, but they shape the relation. Several true programs are rejected because the generic path is stricter than an instance: two copies of a function that inspects its argument are not convertible; a match on a generalised call is not convertible with the call; a block's capture over-approximation rejects a closure inside a statement inside an arm, and a block that takes a place by `&` because an erased term assigns it; changing an index while a dependent field is lent is rejected even when the field's type is the same at every index; large elimination from a Forded proposition is refused at an index where only one constructor is possible; an impossible arm is refused at an unknown index. Each confirms that agreement must be an inequality (generic below instance), as (A2$_{\sqsubseteq}$) and (A3) are, not an equality.

Universes in Lean (Q8). The uniform Lean statement may be unprovable in Lean.

Rules and checker. The Lean statement is about the checker; its correspondence with the rules is tested, not proved. A proof against the checker makes this moot; a paper proof about the rules does not.

10 Open design questions, collected

Open design question Q1 (good ground values). Which ground function values the soundness theorem quantifies over, and what makes calling one safe: a logical relation (with candidates, because of impredicative `Prop`) or definability (closures built by accepted code, with an induction over where they were formed). Section 4.3.

Open design question Q2 (agreement after a block). The ordering (A2$_{\sqsubseteq}$) in which a symbolic $\bot$ left by an over-approximated capture matches any ground value, and its effect on footprints, observations and the consequence that readable places agree. Section 6.5.

Open design question Q3 (stuck equations and `J`). Semantic truth for `Eq` at every type; what, if anything, a stuck equation over $\bot$ asserts; whether ground runs can be stuck at `J`, and if so whether soundness (i) becomes "does not go wrong". Section 6.5.7.

Open design question Q4 (early ends with pairs). A new argument that an early symbolic end of a borrow leaves no reborrow alive that the ground path ends later, now that values have several fields; or a weaker (A3). Section 6.5.

Open design question Q5 (dependent values in the relation). The meaning of an open value in the model and how the two paths' typings of an open field relate, for clause (A7). Section 6.5.

Open design question Q6 (resolution with moved-out contents). Resolution at states where a borrow transiently holds $\bot$, or a walk that only stops at states where every live borrow is whole. Section 6.5.

Open design question Q7 (the typing invariant). An invariant $\mathrm{Inv}_{\mathrm{typ}}$ of states reachable by checked programs: (WF1)–(WF6); every embedded value's declared reading is the reading of its value; no type value is a borrow type produced by computation; the signature is checked. It must be preserved by every machine step and by refinement, and stability (Lemma 26) is then claimed only for states satisfying it. The four statement corrections of Section 9 are evidence that this is needed and of what it must contain.

Open design question Q8 (universes in Lean). Whether the Lean statement keeps one inaccessible per universe level for all levels at once (unprovable in Lean by Gödel if Ochr is as strong as Lean, which is unknown: Ochr has impredicative `Prop` and every $\mathtt{Type}_i$, but no constructor field is a function, so it has no

W-types), or is weakened to bounded universe levels (a family of theorems, one per bound, each at a fixed Lean universe), to a statement relative to an assumed model, or to a statement per program.

Open design question Q9 (a well-founded definition of the model). The interpretation and the injectivity relation R are circular at function types, and "by induction on the type" has nothing smaller to recurse on: codomains are computed by running code, and type variables have no structure. Likely repair: define them together, each type as a set paired with its subset of good elements, and each universe as a set of such pairs; settle together with the gap between truth in the model and the soundness walk. Definition 7.

11 Referee reports

A snapshot. Three independent referees attacked this plan as it stood on 2 November 2026: the first draft, before the set-theory background of Section 4.1.1, the explainer and reminder boxes, open design question Q9 and the notation changes were added. Each was a fresh, read-only agent with a different angle, asked for a statement of the form "this plan will never lead to a final proof of consistency because X", with X a real obstruction rather than a difficulty, and could run probe programs against the checker on main, printing verdicts beside a control that must be rejected. None showed consistency impossible; all three found that the plan *as written* cannot be completed. Their reports are summarised here in the plan's notation, faithfully but condensed; the full texts are in `ochr/docs/reviews/2026-11-02-plan18-*.md`. Section references below are to the current text, which has since absorbed some findings (Q9, and the corrected positivity explainer).

11.1 Foundations and semantics

Statement. The plan defines truth in the set model (Definition 8), which quantifies over arbitrary set-theoretic values, but its only argument for soundness, the walk (Section 4.4), and every lemma beneath it ((P), (C), (Sp), the frame property, stability) speak only of machine values. The checker accepts lemmas whose hypotheses are true in the set model but whose only witnesses are non-computable functions; on those the walk cannot keep its invariants. This blocks the plan's current definitions, not consistency itself, and not the operational strategy as a whole.

Argument. Ochr can state "f decides halting". With `Prog` a type of two-counter machines and `Halts(p)` "p halts from zero", the proposition `ExW(Prog → Nat, Decides)` is true in the set model, because the characteristic function of the halting set witnesses it, but no machine value does: a good closure computes a computable function, and halting is undecidable. The accepted lemma

```
def L3 (e : ExW(Prog → Nat, Decides)) (p : Prog) : Or(Halts(clone(p)), Π(t :
Halts(clone(p))). False) := (
  match e { Wit(w, h) => (
    let hp = h(clone(p));
    match hp { Intro(l, r) => (
      let n = w(clone(p));
      match n { Z => Inl(l(refl)), S m => Inr(r) }) }) })
```

proves excluded middle for halting from that hypothesis. On it the walk breaks. Matching on e binds the data field w to a fresh σ_w and h to type `Decides`(σ_w). The walk's invariant needs `Decides`$(\alpha(\sigma_w))$ true, which every machine closure for $\alpha(\sigma_w)$ makes false; leaving σ_w unrefined instead, the next step must split $n = \lceil \sigma_w(P_0) \rceil$, which is stuck under every machine α and has no head constructor, so the walk cannot choose an arm. The same happens in a closed program: `UseK := K(M)` is accepted, where `M` proves that every decider answers 0 on a program that halts at once, and `K` takes such a proof as its hypothesis. The walk's step for the lemma call needs `M`'s type true for every set-theoretic decider, but the induction hypothesis for `M` covers only machine instances, where no decider exists, so it is vacuous: it would hold equally for an `M` claiming the answer is 1. Filling the

step needs (P), (C), (Sp), the frame property and stability under set-valued valuations, which amounts to a denotational soundness proof.

Evidence. `run`, `Halts`, `Decides`, `ExW`, `M`, `L3`, `K` and `UseK` are all accepted, with every control rejected. A `J` along an equation that is true in the set model but not convertible, `Eq(Nat → Nat, f0, g0)` with `f0(n) := Id0(n)` structurally the identity and `g0(n) := n`, is accepted inside a data function, whose ground run then stops at `J`. Infinitely branching inductives are rejected, through a parameter and through a type function, as is large elimination from a proposition with a data field. No closed proof of `False` was found.

Rescue. (1) Define truth over machine values: a realizability model, that is a logical relation, with Girard-style candidates for impredicative quantification over `Type`. Then `L3′`s hypothesis is false at every instance and `M`'s type holds vacuously; this replaces the set model, the injectivity relation as a set and the inaccessible cardinals, lets conversion soundness follow from the machine-level lemmas, and lives in one Lean universe, which disposes of Q8. (2) Prove the symbolic checker sound denotationally, under set-valued valuations, which makes the walk redundant for consistency. Confidence that the walk cannot prove soundness clause (ii) with set-theoretic truth: about 90%.

Secondary weaknesses.

- Conversion soundness (Lemma 10) has the same gap: at higher types a function's meaning ranges over all set functions, while (P) covers only machine refinements; and the meaning of a codomain at set arguments needs a semantics of code the plan never gives.
- Q3 made concrete: at the instance above, soundness clause (i) is false as stated, since the ground run is stuck at `J`. What an equation's truth means decides it: conversion excludes the instance, extensional equality keeps the problem.
- Q8: Ochr has a universe at every level, so the model needs infinitely many inaccessibles at once; but with no infinitely branching inductives and none above Type_0, Ochr is very plausibly far weaker than Lean, and Gödel blocks this model rather than a proof.
- A cycle: soundness → Q7 → declared readings → decisions → naturality → runs agree → soundness, since a refinement re-runs code whose results keep their declared readings only by subject reduction, part of soundness (i). It is resolvable only as one simultaneous induction, with stability proved per index inside it.
- Q1′s impredicativity worry is misplaced for `Prop`, whose binders can range over the two truth values. The real problem is a Π over `Type` landing in `Prop`: `SelfT := SigP(NoInh)` has good values only if `NoInh := Π(A : Type)(a : A). False` is true, and `NoInh` quantifies over `SelfT`.
- The injectivity relation's induction on types is not well founded (now Q9), and constructor values' recorded parameters are read by the checker as a fallback, harmlessly, contrary to principle (V2).

11.2 The simulation and stability

Statement. The theorem the plan targets is false for the checker it targets, because one of the plan's own premises is false: the worlds rule (Section 5.3) that leaving a split arm keeps the η-refinements of one-constructor unknowns, on the ground that they "follow from the type alone". The checker implements that rule and accepts a closed, hole-free proof of `False`.

Argument. The checker η-expands an abstract σ of a one-constructor type to $\mathsf{C}(\sigma_1, ..., \sigma_k)$, computing the fields' types from σ's type *in the current world*, records the expansion, and reuses it whenever it meets σ again; leaving an arm keeps the record and the types of the unknowns it created. If σ has type $\mathsf{DP}(n)$ and a field's type depends on n, an expansion made in the arm $n := \mathsf{Z}$ stores field unknowns typed at $\mathsf{DP}(\mathsf{Z})$, and the sibling arm $n := \mathsf{S}(m)$ reuses them. The field is dependent, so the dependent-field rules see that its content, of type `Void`, does not fit the telescope's `Nat`, treat the value as open, and type the field by its content; a zero-arm match on it proves anything. So the worlds principle (X1) is violated by the plan's own list of what a world keeps, and (Sp), reduced to (X1)–(X4), is false: at the instance $n = 1$, no refinement can respect the symbolic state, since the field unknown would need a value of `Void`.

Evidence.

```
ochr EtaBoom {
  inductive Void : Type
  inductive Bool := false | true
  def G (n : Nat) (b : Bool) : Type := ( match n { Z => Void, S(m) => Nat } )
  inductive DP (n : Nat) := MkDP(x : Bool, y : G(n, x), z : Nat)
  def Eat (k : Nat) : Unit := ()
  def Bad1 (n : Nat) (d : DP(n)) : False := (
    match n {
      Z => ( let u : Unit = match d { MkDP(x, y, z) => Eat(z) };
             match d { MkDP(x, y, z) => match y {} } ),
      S(m) => ( let u : Unit = match d { MkDP(x, y, z) => Eat(z) };
                match d { MkDP(x, y, z) => match y {} } ),
    }
  )
  def Boom : False := Bad1(S(Z), MkDP(true, Z, Z))
}
```

Every declaration is accepted, `Boom` included, and the control is rejected. The exact hypotheses of `Ochr.Consistency.consistency` hold for this program, and a scratch file proves the theorem's negation. The twin without the first block in the `S` arm, so that no expansion is reused, is rejected ("[Match] on d.y, whose type Nat is not an inductive type"). The coordinator confirmed the acceptance independently on main; the bug, `EtaBoom`, is recorded unfixed in `ochr/docs/17-consistency-status.md`. Confidence: very high.

Rescue. The route survives. Restore the η-records when leaving an arm while still carrying them through private copies within one world, or recompute the field types per world when a record is reused; then prove (Sp) against the checker's actual keep lists item by item, under an explicit invariant that nothing a world keeps names a value created inside the arm or carries a type computed under the arm's refinement, and audit every other kept item (the types of unknowns created in the arm, which also escape through a stuck block's inferred type; the neutrals table; the class cache; the constant cache; the recursion candidates).

Secondary weaknesses.

- N = (P) + (C) + (Sp) is incomplete: some steps consult an unknown without getting stuck or splitting (copy or move decided from an abstract type, η at `Unit`, conversion answering no generically but yes at the instance), and (P) as stated is false: generically $x : T$ is moved, at $T := \mathsf{Unit}$ copied. On the heap design every read in polymorphic code consults the type's unknown, and the heap slice's (P) was proved without moves. The decomposition needs a monotonicity argument with an ordering $\sqsubseteq$.
- M holds only up to $\sqsubseteq$: typed runs decide copy or move from the stored type, untyped runs from the value.
- The heap port's prerequisite that every unknown carries a type refinement never changes is false for field unknowns of value-parameterised types, whose types must change when their index is refined; fixing them at creation would build `EtaBoom` into the heap design.
- (Sp)'s real content is the per-item claims of the keep lists, which the plan should list as open obligations.

11.3 The structure of the argument and its targets

Statement. No strict impossibility, but the notion of truth and the soundness theorem do not meet, and fixing it changes the model. Truth is set-theoretic, soundness covers only the countably many machine-valued ground instances, and accepted programs prove propositions quantifying over `Type` and over functions, whose truth concerns every element of an uncountable set. The meaning of a function value or codomain at such an element is never defined, so "proof terms have true types" cannot be discharged. The pieces fail to compose; this is not a proof that Ochr is inconsistent.

Evidence. `L (A : Type) (a : A) : Eq(A, a, a) := refl`, `M := L` at type $\Pi(A : \mathsf{Type})(a : A).\ \mathsf{Eq}(A, a, a)$, `H` taking such a function as hypothesis, `K := H(L)`, and their variants over functions $\Pi(f : \Pi(n : \mathsf{Nat}).\ \mathsf{Nat})$ are all accepted. To check `K`, the walk must show `H`'s hypothesis true for every set A, which nothing supplies; Q1′s two candidates for "good" are both defined by runs at definable arguments.

Rescue. (A) Relativise truth to good, definable values (a term or realizability model); the walk survives, at the cost of a truth-value argument for `Prop` binders, an extensionality lemma that makes Q3 mandatory (a `J` at endpoints equal in meaning but not convertible computes on one and sticks on the other), and conversion soundness re-proved. (B) Give Ochr code a denotational semantics in the set model, borrows and closing off included, with adequacy against the machine: roughly a second soundness proof, and the object the model already silently presupposes. Confidence that the gap is real as written: high; that it cannot be rescued: low.

Secondary weaknesses.

- The target statement holds up, but about 80 unchecked array accesses in the checker, which return a default value when out of range, are a missing proof obligation.
- Restricting stability to reachable states creates a cycle: a definition is added to the signature before its body is checked, and refinement re-runs its unchecked code (a probe shows an error raised in one arm by running another arm's unchecked body). Stability must be fused into the walk, relative to β, and soundness (i) must include type preservation.
- The Lean `Conjecture2` is false as stated: at the reachable generic state of the accepted `G (n x : Nat) : Unit := ((match n { Z => (let y = x; ()), S _ => () }); ())`, refining the generic result leaves `x` moved out while the direct run at $n := 1$ keeps it, and no renaming turns $\bot$ into 1. So `theorem stability` is a `sorry` proof of a false proposition, as recorded in `ochr/docs/17-consistency-status.md`; it must be restated with an ordering.
- The heap-slice results put the substance in their hypotheses: locality is assumed, not proved, and measured on 1,166 runs; (P) needs renaming invariance as soon as a refinement allocates.
- Two machines with no bridge: the agreement clauses and resolution are about moving borrows, while (P), the frame property and (C) are proved on the heap design, where neither exists; after the port the checker would no longer implement the paper's rules.
- The recommended order never produces a Lean proof of `Ochr.Consistency.consistency`: nothing mechanises the model, conversion soundness or the walk, and nothing links acceptance to a derivation.
- Inaccessible cardinals may be unnecessary, since Ochr has no W-types, which would make Q8 moot (unverified).